

SLAM Handbook
从定位与建图到空间智能

第 1、2 章中文翻译

2026 年 8 月 9 日

版权与翻译说明

本书原著由 Luca Carlone、Ayoung Kim、Timothy Barfoot、Daniel Cremers 和 Frank Dellaert 编辑, © Cambridge University Press。

本 PDF 是 SLAM Handbook 第 1、2 章的中文翻译排版版本。原作者、编辑、出版方及原项目的版权和署名信息均予以保留。中文翻译内容仅在获得相应授权的范围内公开使用。

目录

版权与翻译说明	i
第一章 SLAM 的因子图	1
历史注记	1
1.1 用因子图直观表示 SLAM	1
1.1.1 一个玩具示例	2
1.1.2 因子图视角	3
1.1.3 将因子图作为一种语言	5
1.2 从 MAP 推断到最小二乘	6
1.2.1 用因子图进行 MAP 推断	6
1.2.2 指定概率密度	7
1.2.3 非线性最小二乘	8
1.3 求解线性最小二乘	9
1.3.1 线性化	9
1.3.2 将 SLAM 写成线性最小二乘	10
1.3.3 用矩阵分解求解最小二乘	10
1.4 非线性优化	11
1.4.1 最速下降	11
1.4.2 高斯—牛顿	12
1.4.3 Levenberg—Marquardt	12
1.4.4 Dogleg 最小化	12
1.5 因子图与稀疏性	13
1.5.1 稀疏雅可比矩阵及其因子图	13
1.5.2 稀疏信息矩阵及其图	13
1.5.3 稀疏分解	14
1.6 消元	15
1.6.1 变量消元算法	15
1.6.2 线性高斯消元	18
1.6.3 作为贝叶斯网络的稀疏 Cholesky 因子	19
1.7 增量式 SLAM	21
1.7.1 贝叶斯树	21
1.7.2 更新贝叶斯树	22
1.7.3 增量平滑与建图	22
1.8 延伸阅读与近期趋势	23

第二章 高级状态变量表示	24
2.0.1 流形上的优化	24
2.0.2 旋转与位姿	25
2.0.3 矩阵 Lie 群	25
2.0.4 Lie 群优化	27
2.0.5 不确定性与 Lie 群	28
2.0.6 Lie 群的其他工具	29
2.0.6.1 $\oplus$ 与 $\ominus$ 算子	29
2.0.6.2 逆	29
2.0.6.3 伴随	29
2.0.6.4 雅可比	30
2.1 连续时间轨迹	30
2.1.1 样条	31
2.1.2 从参数化到非参数化	32
2.1.3 高斯过程	33
2.1.4 Lie 群上的样条与高斯过程	34
2.1.4.1 Lie 群上的样条	34
2.1.4.2 Lie 群上的高斯过程	36
2.2 延伸阅读与近期趋势	37

第一章 SLAM 的因子图

作者： Frank Dellaert、Michael Kaess、Timothy Barfoot

本章介绍因子图，并在高斯先验和高斯测量噪声的条件下，建立最大后验（maximum a posteriori, MAP）推断与最小二乘之间的联系。我们关注 SLAM 后端，即前端已经提取测量并完成数据关联之后的部分。首先，我们使用因子图对 SLAM 问题进行可视化（第 1.1 节）；随后说明 MAP 推断如何导出最小二乘优化（第 1.2 节）。接着讨论线性优化（第 1.3 节）和非线性优化（第 1.4 节）的求解方法，并进一步明确稀疏性、因子图和贝叶斯网络之间的联系（第 1.5 节）。然后介绍变量消元算法及其图模型解释（第 1.6 节），最后据此构建贝叶斯树以及增量平滑与建图（iSAM）算法（第 1.7 节）。本章最后讨论延伸阅读与近期趋势（第 1.8 节）。

历史注记

SLAM 的平滑方法不仅考虑机器人当前时刻的位置，还考虑截至当前时刻的完整机器人轨迹。许多作者只研究机器人轨迹的平滑问题^[172, 701, 700, 420, 594, 312]，这类方法如今称为**基于位姿的 SLAM**（pose-based SLAM）。它尤其适用于激光测距仪等传感器，因为这类传感器会在相邻机器人位姿之间产生两两约束。

更一般地，可以考虑完整 SLAM 问题^[1088]，即在最优意义下，同时估计全部传感器位姿以及环境中所有特征的参数。从计算角度看，这种基于优化的平滑方法具有明显优势：(a) 与基于滤波的协方差矩阵或信息矩阵不同，后两者都会随时间推移逐渐变成完全稠密的矩阵^[849, 1087]，而平滑对应的信息矩阵保持稀疏；(b) 在典型建图场景中（也就是机器人并非反复穿行于一个很小的环境），该矩阵能够更紧凑地表示地图协方差结构。由此，在 2000—2005 年间，大量工作开始将这些思想应用于 SLAM^[287, 340, 339, 1088]。

基于平方根的平滑与建图（square-root smoothing and mapping, SAM），也称为“因子图方法”，是在^[249, 254]中提出的，其依据是：信息矩阵或测量雅可比矩阵可以分别通过稀疏 Cholesky 分解或 QR 分解进行高效分解。这样便得到一个平方根信息矩阵，可立即用于求取最优机器人轨迹和地图。在序贯估计文献中，对信息矩阵进行分解被称为**平方根信息滤波**（square-root information filtering, SRIF）；该方法于 1969 年为 JPL 的水手 10 号（Mariner 10）金星探测任务而发展^[83]。使用平方根能够得到更加准确、稳定的算法。引用 Maybeck^[742] 的话说：“许多实践者以充分的逻辑依据主张：平方根滤波器应始终优先于标准的卡尔曼滤波递推。”

下面我们将详细讨论：因子图为何是表示 SLAM 问题固有稀疏性的自然工具；将矩阵进行（稀疏）分解并得到矩阵平方根为何是求解这类问题的核心；以及这些内容最终如何与更一般的变量消元算法联系起来。本章的大部分内容是 Dellaert 等人^[255] 长篇文章的压缩版。

1.1 用因子图直观表示 SLAM

本节首先通过一个玩具示例及其因子图表示，引入因子图，直观展示 SLAM 问题的稀疏性。随后说明多种不同形式的 SLAM 都可以采用这种表示，即使面对更大规模的问题，其稀疏结构也通常一目了然。

1.1.1 一个玩具示例

我们先考察一个简单的 SLAM 场景，以说明如何构造因子图。图 1.1 给出了该问题结构的示意图。机器人依次经过三个位姿 p_1 、 p_2 和 p_3 ，并对两个路标 ℓ_1 和 ℓ_2 进行方位观测。为了将解锚定在空间中，假设在第一个位姿 p_1 上还有一个绝对位置/方向测量。没有这一测量，就无法获得绝对位置的信息，因为方位测量都是相对的。¹

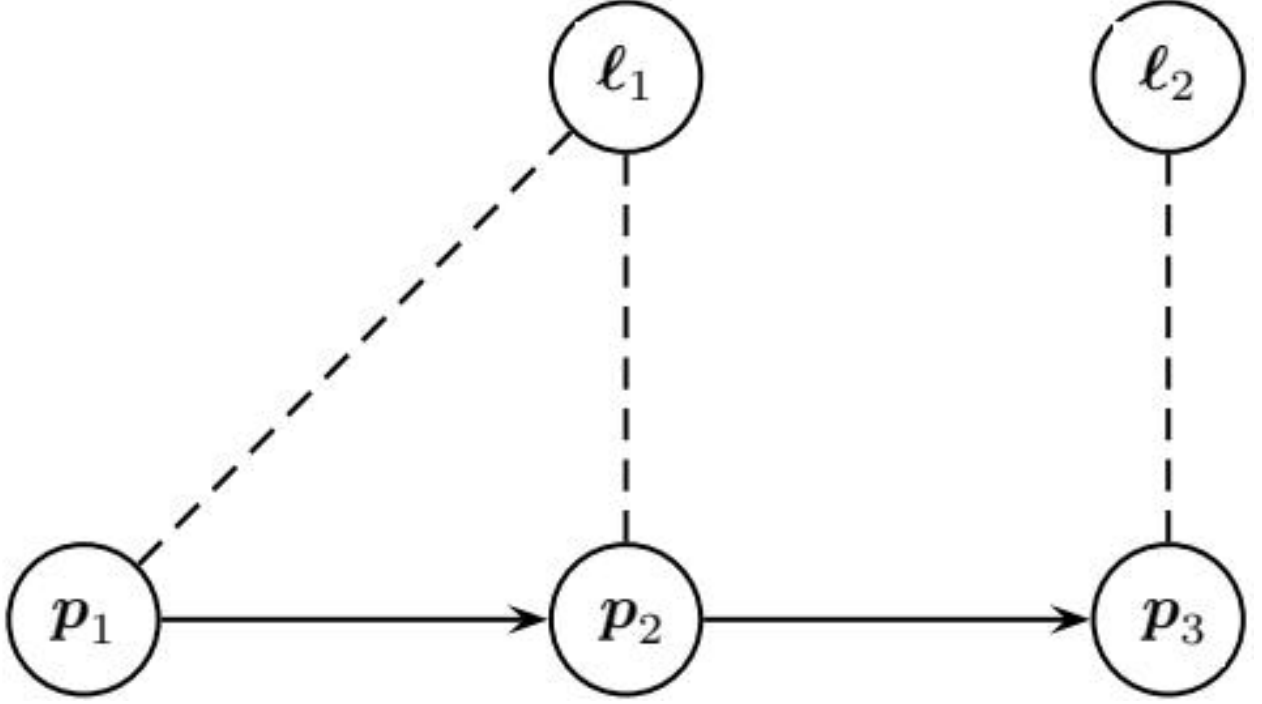


图 1.1: 图 1.1 三个位姿和两个路标构成的玩具同步定位与建图 (SLAM) 示例。图中用箭头示意机器人运动，用虚线表示方位测量。

由于测量存在不确定性，我们不可能恢复世界的真实状态，但可以得到一个关于测量能够推断出的内容的概率描述。在贝叶斯概率框架中，我们使用概率论的语言，为不确定事件赋予主观的可信程度。具体而言，对未知变量 $\mathbf{x}$ 定义概率密度函数 (probability density function, PDF) $p(\mathbf{x})$ 。PDF 是满足下式的非负函数：

$$\int p(\mathbf{x}) d\mathbf{x} = 1, \quad (1.1)$$

这就是全概率公理。在图 1.1 的简单示例中，状态 $\mathbf{x}$ 为

$$\mathbf{x} = \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ \ell_1 \\ \ell_2 \end{bmatrix}, \quad (1.2)$$

也就是将各个未知量依次堆叠起来。

在 SLAM 中，给定一组观测测量 $\mathbf{z}$ 后，我们希望刻画关于未知量 $\mathbf{x}$ 的知识；在本例中，未知量包括机器人位姿和未知的路标位置。用贝叶斯概率的语言，这就是条件密度或后验密度：

$$p(\mathbf{x}|\mathbf{z}), \quad (1.3)$$

获得这样的描述称为**概率推断**。在此之前，必须先为感兴趣的变量以及它们如何产生（不确定的）测量指定概率模型。这正是概率图模型发挥作用的地方。

概率图模型利用概率分布中的结构，为复杂概率密度提供紧凑的描述机制^[592]。贝叶斯网络可能是最广为人知的图模型：它由变量节点组成，每个变量节点都关联一个先验概率密度或条件概率密度。图 1.2 展示了图 1.1 玩具示例对应的贝叶斯网络，其中明确给出了依赖于第一个位姿 p_1 的初始测量 z_1 ，以及 $z_2 \dots z_4$ 四个方位测量，每个测量都同时与一个位姿和一个路标相关。按照惯例，贝叶斯网络中已知量用方形节点表示，如图所示。不过，贝叶斯网络虽然非常适合建模，下面我们还要引入另一种更适合优化的图模型；它只关注问题中的未知变量。

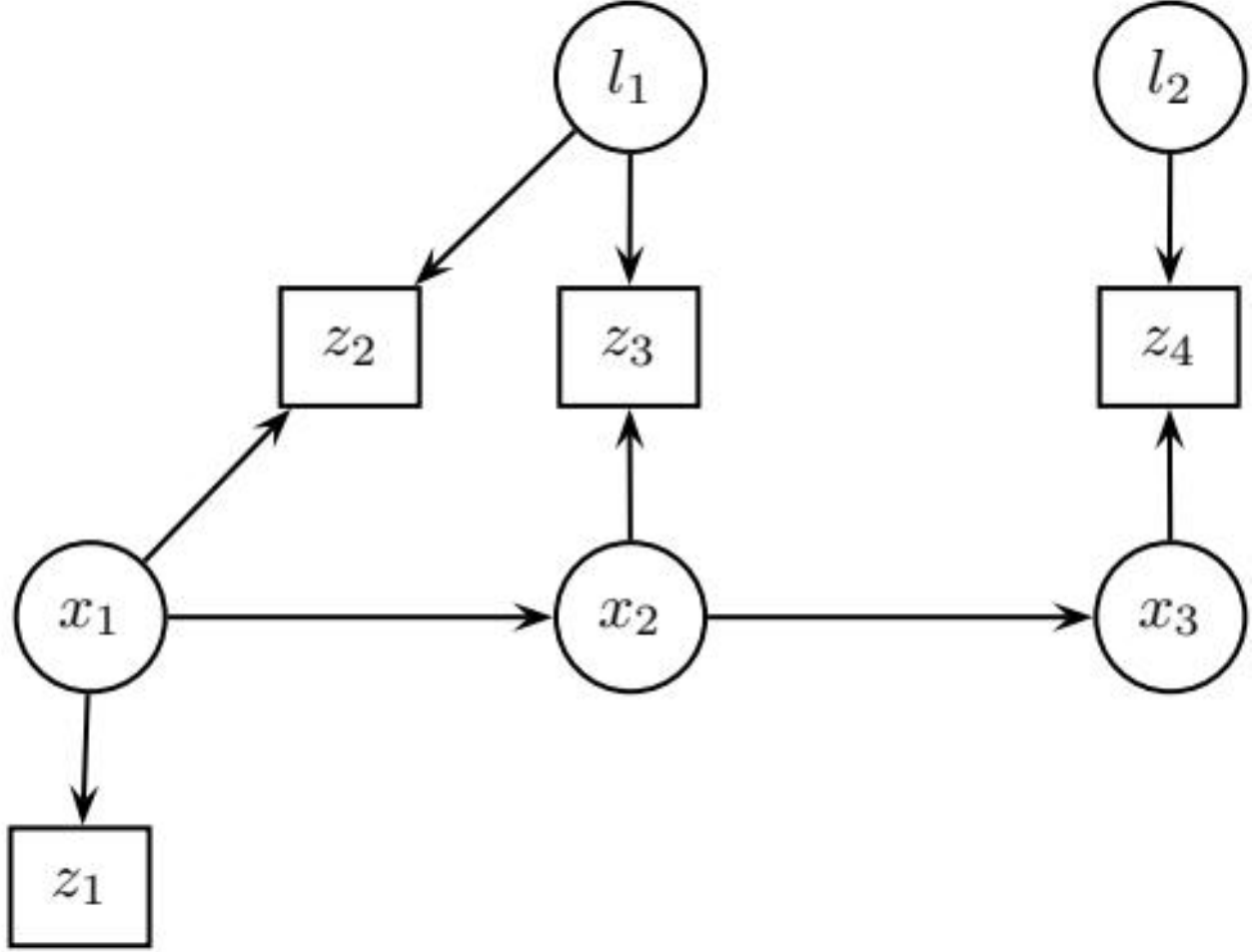


图 1.2: 图 1.1 示例对应的贝叶斯网络，其中将已知测量 $z_1 \dots z_4$ 明确表示为方形节点。

1.1.2 因子图视角

由于局部性，高维概率密度通常可以分解为许多因子的乘积，每个因子只是定义在一个更小域上的概率密度。因子图是一类概率图模型，它允许我们将任意密度表示为因子的乘积。

为了说明因子图的动机，考虑如何对上述玩具 SLAM 示例执行推断。根据贝叶斯定律，后验 $p(\mathbf{x}|\mathbf{z})$ (式 1.3) 可写为 $p(\mathbf{x}|\mathbf{z}) \propto p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$ ，于是

$$p(\mathbf{x}|\mathbf{z}) \propto p(\mathbf{p}_1)p(\mathbf{p}_2|\mathbf{p}_1)p(\mathbf{p}_3|\mathbf{p}_2) \quad (1.4a)$$

$$\times p(\ell_1)p(\ell_2) \quad (1.4b)$$

$$\times p(\mathbf{z}_1 | \mathbf{p}_1) \quad (1.4c)$$

$$\times p(\mathbf{z}_2 | \mathbf{p}_1, \ell_1) p(\mathbf{z}_3 | \mathbf{p}_2, \ell_1) p(\mathbf{z}_4 | \mathbf{p}_3, \ell_2). \quad (1.4d)$$

这里假设位姿轨迹服从典型的马尔可夫链生成模型。每个因子都代表关于未知量 $\mathbf{x}$ 的一部分信息。

为了可视化这种分解，我们使用因子图。图 1.3 展示了该示例对应的因子图：所有未知状态 $\mathbf{x}$ （包括位姿和路标）都有一个对应节点。测量是已知量，因此不显式表示，也不是我们关注的对象。在因子图中，我们显式引入另一类节点，用来表示后验 $p(\mathbf{x} | \mathbf{z})$ 中的每个因子。图中每个小黑节点表示一个因子；更重要的是，它只连接到该因子所依赖的状态变量。例如，因子 $\phi_9(\mathbf{p}_3, \ell_2)$ 只连接到变量节点 $\mathbf{p}_3$ 和 ℓ_2 。具体地，有

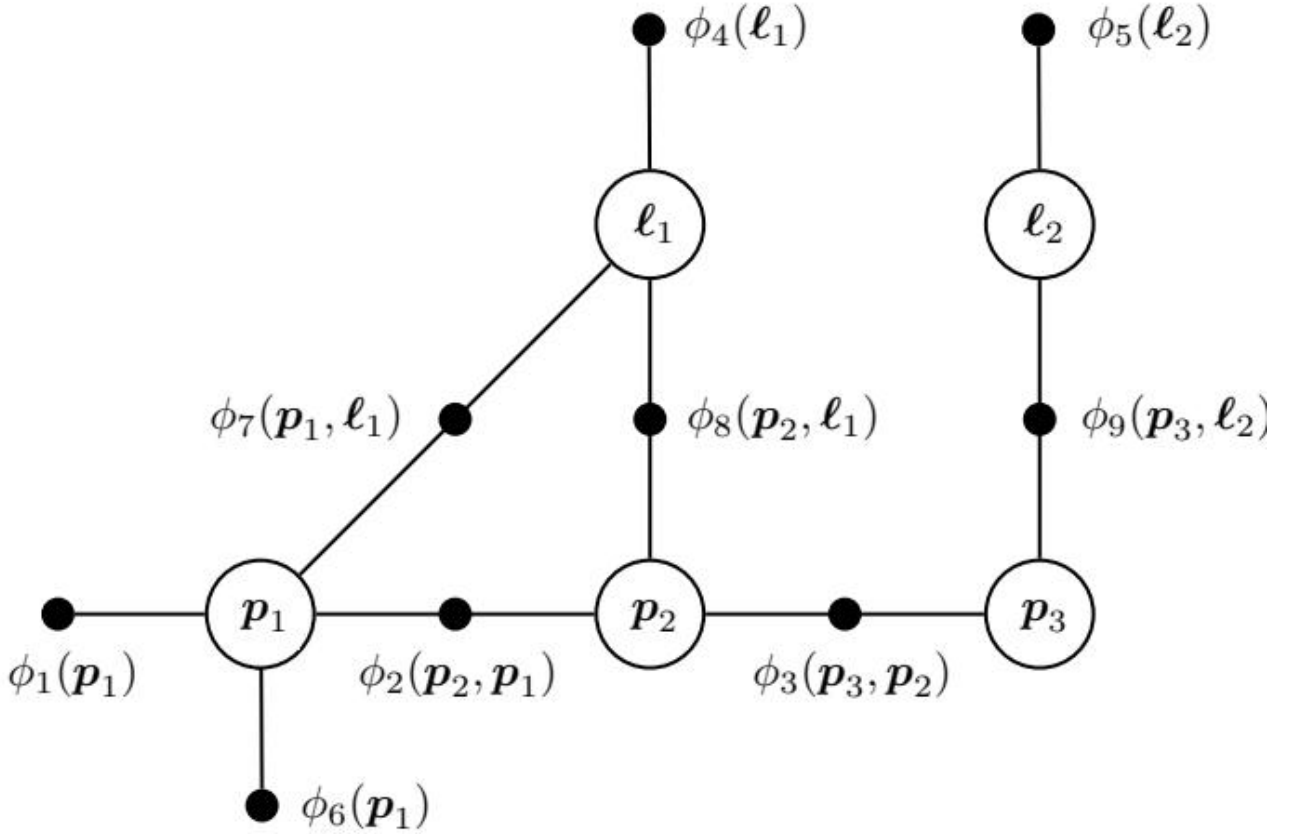


图 1.3: 图 1.3 图 1.1 示例得到的因子图。

$$\phi(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \ell_1, \ell_2) = \phi_1(\mathbf{p}_1) \phi_2(\mathbf{p}_2, \mathbf{p}_1) \phi_3(\mathbf{p}_3, \mathbf{p}_2) \quad (1.5a)$$

$$\times \phi_4(\ell_1) \phi_5(\ell_2) \quad (1.5b)$$

$$\times \phi_6(\mathbf{p}_1) \quad (1.5c)$$

$$\times \phi_7(\mathbf{p}_1, \ell_1) \phi_8(\mathbf{p}_2, \ell_1) \phi_9(\mathbf{p}_3, \ell_2), \quad (1.5d)$$

因子与式 (1.4a) — (1.4d) 中原始概率密度之间的对应关系应当是显而易见的。

因子值只需与相应概率密度成正比即可：任何与状态变量无关的归一化常数都可以省略，不会造成影响。在本例中，上述因子要么来自先验，例如 $\phi_1(p_1) \propto p(p_1)$ ；要么来自测量，例如 $\phi_9(p_3, \ell_2) \propto p(z_4|p_3, \ell_2)$ 。虽然测量变量 $z_1 \dots z_4$ 没有在因子图中显式显示，但这些因子都隐式地以它们为条件。有时为了使这一点更加明确，也可以将因子写成（例如） $\phi_9(p_3, \ell_2; z_4)$ ，甚至写成 $\phi_{z_4}(p_3, \ell_2)$ 。

1.1.3 将因子图作为一种语言

因子图除了为推断提供形式化基础之外，还能帮助我们可视化多种不同形式的 SLAM 问题、洞察问题结构，并充当一种通用语言，使不同团队的实践者能够在统一的概念下交流。图 1.3 中的每个因子都可以看作一个涉及其所连接变量的方程。通常，方程数量远多于未知量，因此必须量化先验信息和测量中的不确定性。这将导出最小二乘形式，从而适当地融合多个信息源的信息。

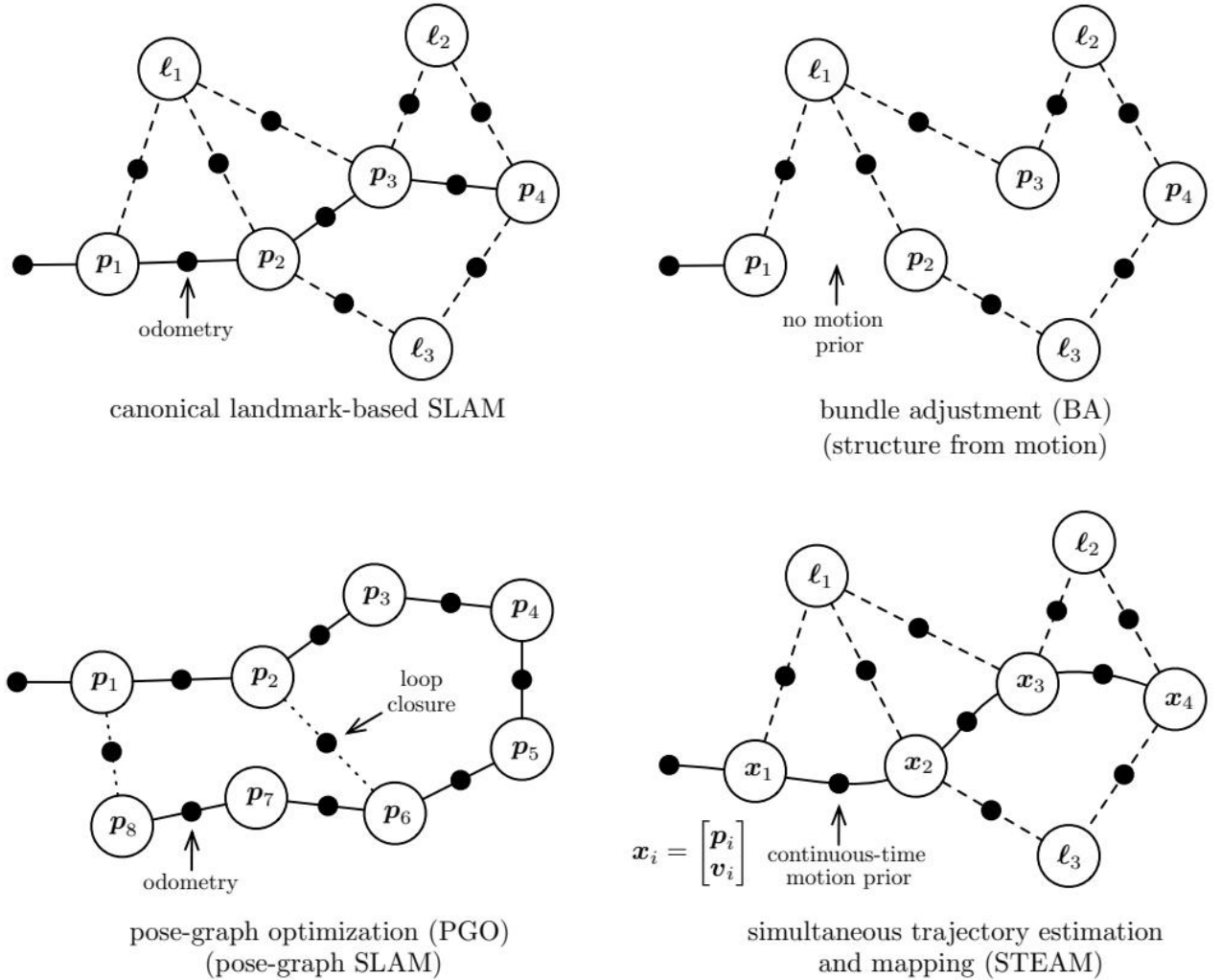


图 1.4: 图 1.4 几种都可以用因子图视角表示的 SLAM 问题变体。典型的基于路标的 SLAM 同时包含位姿变量和路标变量；从位姿观测路标，并且位姿之间通常存在基于里程计的运动先验。束调整（BA）与之相同，但不包含运动先验。位姿图优化（PGO）不含路标变量，但在位姿之间增加了回环测量。最后，同时轨迹估计与建图（STEAM）与基于路标的 SLAM 类似，只是将位姿替换为更高阶状态，并采用平滑的连续时间运动先验。

多种形式的 SLAM 问题都可以很容易地用因子图表示。图 1.1 是基于路标的 SLAM 示例，因为它同时包含位姿变量和路标变量。图 1.4 给出了其他几种变体，包括束调整（bundle adjustment, BA；与基于路标的 SLAM 相同，但没有运动模型）、位姿图优化（pose-graph optimization, PGO；没有路标

变量，但包含位姿之间的回环约束)，以及同时轨迹估计与建图 (simultaneous trajectory estimation and mapping, STEAM; 在位姿中加入速度等导数项)。

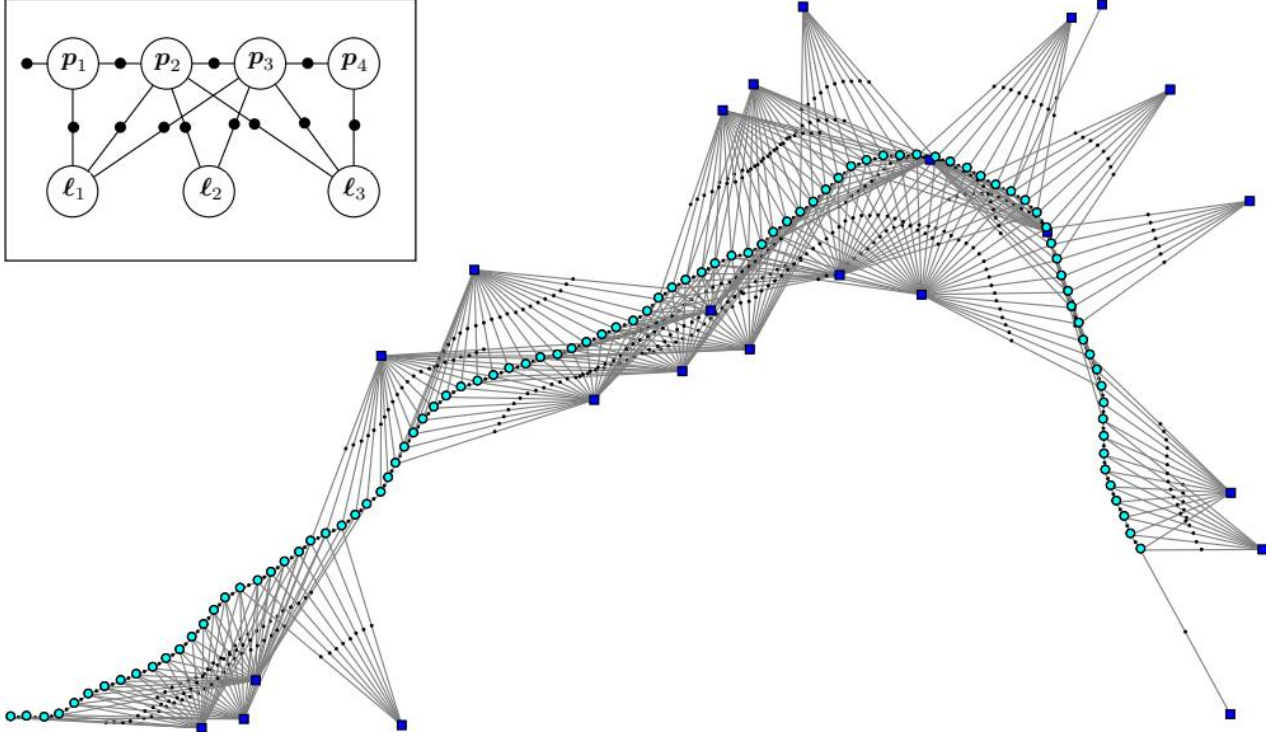


图 1.5: 图 1.5 一个规模更大的模拟 SLAM 示例因子图。

比图 1.1 的玩具示例更接近实际的基于路标的 SLAM 问题，其因子图可能类似图 1.5。该图通过模拟一个在平面内运动约 100 个时间步、同时观测路标的二维机器人生成。为便于可视化，每个位姿和路标都按其真实二维位置绘制。可以看到，里程计因子形成了显著的链状骨架，而在两侧则有二元²似然因子连接到约 20 个路标。在这类 SLAM 问题中，除先验之外，所有因子通常都是非线性的。

检查因子图可以揭示大量结构信息，帮助我们理解某个具体的 SLAM 问题实例。首先，有些路标拥有大量测量，我们预计它们会被非常准确地确定；另一些路标与图的联系很弱，因此估计结果也会更不确定。例如，右下方孤立的路标只有一个关联测量。

1.2 从 MAP 推断到最小二乘

最大后验 (MAP) 推断是确定未知量 $\mathbf{x}$ 取值的过程，使其与不确定测量和先验中包含的信息尽可能一致。在实际情况下，我们并不知道路标的真实位置，也不知道机器人随时间变化的位姿，尽管在许多应用中可能有一个较好的初始估计。下面将说明：在采用高斯测量噪声模型和高斯先验的条件下，MAP 推断对应的优化问题就是我们熟悉的非线性最小二乘问题。

1.2.1 用因子图进行 MAP 推断

给定测量 $\mathbf{z}$ ，我们关注位姿和/或路标等未知状态变量 $\mathbf{x}$ 。对于这些未知状态变量，最常用的估计量是最大后验 (MAP) 估计，其名称源于它最大化了给定测量 $\mathbf{z}$ 时状态 $\mathbf{x}$ 的后验密度 $p(\mathbf{x}|\mathbf{z})$ ：

$$\mathbf{x}^{\text{MAP}} = \arg \max_{\mathbf{x}} p(\mathbf{x}|\mathbf{z}) \quad (1.6a)$$

$$= \arg \max_{\mathbf{x}} \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} \quad (1.6b)$$

$$= \arg \max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x})p(\mathbf{x}) \quad (1.6c)$$

上式中的第二个等式是贝叶斯定律，它将后验表示为测量密度 $p(\mathbf{z}|\mathbf{x})$ 与状态先验 $p(\mathbf{x})$ 的乘积，再用 $p(\mathbf{z})$ 进行适当归一化。第三个等式省略了 $p(\mathbf{z})$ ，因为它与 $\mathbf{x}$ 无关，不会影响求 $\arg \max$ 的结果。换言之，式 (1.6c) 告诉我们：MAP 估计使似然 $p(\mathbf{z}|\mathbf{x})$ 与先验 $p(\mathbf{x})$ 的乘积最大。

我们使用因子图表示未归一化后验 $p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$ 。形式上，因子图 $\mathbf{F} = (\mathcal{U}, \mathcal{V}, \mathcal{E})$ 是一个二部图，包含两类节点：因子 $\phi_i \in \mathcal{U}$ 和变量 $\mathbf{x}_j \in \mathcal{V}$ 。边 $e_{ij} \in \mathcal{E}$ 始终连接因子节点和变量节点。与因子 ϕ_i 相邻的变量节点集合记作 $\mathcal{X}(\phi_i)$ ；我们用 $\Delta_{\mathbf{x}_i}$ 表示对该集合的一次赋值。按照这些定义，因子图 $\mathbf{F}$ 将全局函数 $\phi(\mathbf{x})$ 定义为

$$\phi(\mathbf{x}) = \prod_i \phi_i(\mathbf{x}_i). \quad (1.7)$$

换言之，独立关系由因子图中的边 e_{ij} 编码；每个因子 ϕ_i 只依赖于其邻接集合 $\mathcal{X}(\phi_i)$ 中的变量 $\Delta_{\mathbf{x}_i}$ 。

本章其余部分将说明，如何通过因子图中的未知变量进行优化，找到最优赋值，即 MAP 估计。对于任意因子图，MAP 推断归结为最大化所有因子图势函数的乘积（式 1.7）：

$$\mathbf{x}^{\text{MAP}} = \arg \max_{\mathbf{x}} \phi(\mathbf{x}) \quad (1.8a)$$

$$= \arg \max_{\mathbf{x}} \prod_i \phi_i(\mathbf{x}_i). \quad (1.8b)$$

接下来需要推导因子 $\phi_i(\mathbf{x}_i)$ 的具体形式，而这在很大程度上取决于测量模型 $p(\mathbf{z}|\mathbf{x})$ 和先验密度 $p(\mathbf{x})$ 的建模方式。下面将详细讨论。

1.2.2 指定概率密度

上述密度 $p(\mathbf{z}|\mathbf{x})$ 和 $p(\mathbf{x})$ 的具体形式高度依赖于应用和所使用的传感器。最常用的密度是多元高斯密度，其概率密度为

$$\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2\right), \quad (1.9)$$

其中， $\boldsymbol{\mu} \in \mathbb{R}^n$ 为均值， $\boldsymbol{\Sigma}$ 为 $n \times n$ 协方差矩阵，并且

$$\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2 \triangleq (\boldsymbol{\theta} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{\theta} - \boldsymbol{\mu}) \quad (1.10)$$

表示平方马氏距离。归一化常数 $\sqrt{|2\pi\boldsymbol{\Sigma}|} = (2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}$ ，其中 $|\cdot|$ 表示矩阵行列式，它保证多元高斯密度在其定义域上的积分为 1。

未知量的先验通常用高斯密度指定；在许多情况下，将测量建模为受到零均值高斯噪声污染既合理又方便。例如，给定位姿 p 到给定路标 ℓ 的方位测量³ 可建模为

$$\mathbf{z} = \mathbf{h}(p, \ell) + \boldsymbol{\eta}, \quad (1.11)$$

其中 $\mathbf{h}(\cdot)$ 是测量预测函数，噪声 $\boldsymbol{\eta}$ 服从协方差为 $\boldsymbol{\Sigma}_R$ 的零均值高斯密度。于是，关于测量 $\mathbf{z}$ 的条件密度 $p(\mathbf{z}|\mathbf{p}, \ell)$ 为

$$p(\mathbf{z}|\mathbf{p}, \ell) = \mathcal{N}(\mathbf{z}; \mathbf{h}(\mathbf{p}, \ell), \Sigma_R) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2}\|\mathbf{z} - \mathbf{h}(\mathbf{p}, \ell)\|_{\Sigma_R}^2\right). \quad (1.12)$$

在实际机器人应用中, 测量函数 $h(\cdot)$ 往往是非线性的。不过, 尽管它取决于所使用的传感器和 SLAM 前端, 但通常并不难理解或写出。二维方位测量的测量函数就是

$$\mathbf{h}(\mathbf{p}, \ell) = \text{atan2}(\ell_y - p_y, \ell_x - p_x), \quad (1.13)$$

其中 ℓ_x, ℓ_y 是路标的 x, y 坐标, p_x, p_y 是位姿的 x, y 坐标, atan2 是众所周知的双参数反正切函数。因此, 最终的概率测量模型 $p(\mathbf{z}|\mathbf{p}, \ell)$ 为

$$p(\mathbf{z}|\mathbf{p}, \ell) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2}\|\mathbf{z} - \text{atan2}(\ell_y - p_y, \ell_x - p_x)\|_{\Sigma_R}^2\right). \quad (1.14)$$

需要注意的是, 我们并不总是假设测量噪声为高斯噪声。例如, 为了应对偶发的数据关联错误, 许多作者提出使用具有更重尾部的鲁棒测量密度; 第 3 章将讨论这些方法。

并非所有涉及的概率密度都来自测量。例如, 在玩具 SLAM 问题中, 轨迹先验 $p(\mathbf{x})$ 由先验 $p(\mathbf{p}_1)$ 和条件密度 $p(\mathbf{p}_{t+1}|\mathbf{p}_t)$ 构成, 用于描述在给定已知控制输入 $\mathbf{u}_t$ 的条件下机器人所遵循的概率运动模型。实践中, 我们通常采用条件高斯假设:

$$p(\mathbf{p}_{t+1}|\mathbf{p}_t, \mathbf{u}_t) = \frac{1}{\sqrt{|2\pi\Sigma_Q|}} \exp\left(-\frac{1}{2}\|\mathbf{p}_{t+1} - \mathbf{g}(\mathbf{p}_t, \mathbf{u}_t)\|_{\Sigma_Q}^2\right), \quad (1.15)$$

其中 $\mathbf{g}(\cdot)$ 是运动模型, Σ_Q 是维度适当的协方差矩阵; 例如, 对于在平面内运动的机器人, 它是 3×3 矩阵。

我们经常没有已知的控制输入 $\mathbf{u}_t$, 而是测量机器人如何运动, 例如通过里程计测量 $\mathbf{o}_t$ 。若假设里程计只是测量位姿差, 并受到协方差为 Σ_S 的高斯噪声影响, 则有

$$p(\mathbf{o}_t|\mathbf{p}_{t+1}, \mathbf{p}_t) = \frac{1}{\sqrt{|2\pi\Sigma_S|}} \exp\left(-\frac{1}{2}\|\mathbf{o}_t - (\mathbf{p}_{t+1} - \mathbf{p}_t)\|_{\Sigma_S}^2\right). \quad (1.16)$$

如果同时拥有已知控制输入 $\mathbf{u}_t$ 和里程计测量 $\mathbf{o}_t$, 就可以将式 (1.15) 与式 (1.16) 结合起来。

对于在三维空间中运行的机器人, 需要使用稍微复杂一些的工具, 为 $\text{SE}(3)$ 这类非线性流形指定概率密度; 下一章将对此进行讨论。

1.2.3 非线性最小二乘

现在说明, 对于采用上述高斯噪声模型的 SLAM 问题, MAP 推断等价于求解非线性最小二乘问题。假设所有因子都与多元高斯分布成正比, 即

$$\phi_i(\mathbf{x}_i) \propto \exp\left(-\frac{1}{2}\|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2\right). \quad (1.17)$$

这既包括简单的高斯先验因子, 也包括由零均值正态分布噪声污染的测量所导出的似然因子。将式 (1.17) 代入式 (1.8b), 取负对数并去掉因子 $1/2$, 便可转而最小化一组非线性最小二乘项之和:

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2. \quad (1.18)$$

最小化这个目标函数, 就是将多个由测量得到的因子以及可能存在的多个先验结合起来, 执行传感器融合, 从而确定未知量的 MAP 解。

一个重要而不太显然的观察是, 式 (1.18) 中的因子通常只描述所涉及未知变量 $\Delta\mathbf{x}_i$ 的部分概率。除简单的先验因子外, 测量 $\mathbf{z}_i$ 的维度通常低于未知量 $\Delta\mathbf{x}_i$ 的维度。在这种情况下, 单个因子会为 $\mathbf{x}_i$ 定

义域中的一个无限子集赋予相同的似然。例如，相机图像中的一个二维测量，与一整条能够投影到该图像位置的三维点射线相容。

尽管函数 $\mathbf{h}_i$ 是非线性的，但如果有一个足够好的初始猜测，本章讨论的非线性优化方法通常能够收敛到式 (1.18) 的全局最小值。不过需要提醒的是，式 (1.18) 的目标函数是非凸的，因此当初始猜测较差时，无法保证不会陷入局部最小值。这促成了所谓的**可证明最优求解器**，第 6 章将对此进行介绍。下面我们先关注局部方法，而不是全局求解器；首先考虑更容易的线性化问题。

1.3 求解线性最小二乘

在处理更困难的非线性最小二乘之前，本节首先说明如何将问题线性化，展示线性化如何导出线性最小二乘问题，并回顾如何通过矩阵分解高效求解相应的正规方程。关于这些方法的经典参考是 Golub 和 Van Loan 的著作^[389]。

1.3.1 线性化

可以使用简单的泰勒展开，将非线性最小二乘目标函数（式 1.18）中的所有测量函数 $\mathbf{h}_i(\cdot)$ 线性化：

$$\mathbf{h}_i(\mathbf{x}_i) = \mathbf{h}_i(\mathbf{x}_i^0 + \boldsymbol{\delta}_i) \approx \mathbf{h}_i(\mathbf{x}_i^0) + \mathbf{H}_i \boldsymbol{\delta}_i, \quad (1.19)$$

其中，测量雅可比 $\mathbf{H}_i$ 定义为在给定线性化点 $\mathbf{x}_i^0$ 处 $\mathbf{h}_i(\cdot)$ 的（多元）偏导数：

$$\mathbf{H}_i \triangleq \left. \frac{\partial \mathbf{h}_i(\mathbf{x}_i)}{\partial \mathbf{x}_i} \right|_{\mathbf{x}_i^0}, \quad (1.20)$$

而 $\boldsymbol{\delta}_i \triangleq \mathbf{x}_i - \mathbf{x}_i^0$ 是状态更新向量。这里假设 $\Delta \mathbf{x}_i$ 位于向量空间中，或者等价地，可以用向量表示。当 $\mathbf{x}$ 中的一些未知状态表示三维旋转或其他更复杂的流形类型时，情况并不总是如此。第 2 章将重新讨论这个问题。

将泰勒展开（式 1.19）代入非线性最小二乘表达式（式 1.18），得到关于状态更新向量 $\boldsymbol{\delta}$ 的线性最小二乘问题：

$$\boldsymbol{\delta}^* = \arg \min_{\boldsymbol{\delta}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0) - \mathbf{H}_i \boldsymbol{\delta}_i\|_{\boldsymbol{\Sigma}_i}^2 \quad (1.21a)$$

$$= \arg \min_{\boldsymbol{\delta}} \sum_i \|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \boldsymbol{\delta}_i\|_{\boldsymbol{\Sigma}_i}^2, \quad (1.21b)$$

其中， $\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)$ 是线性化点处的预测误差，即实际测量与预测测量之间的差。上式中的 $\boldsymbol{\delta}^*$ 表示局部线性化问题的解。

通过一个简单的变量替换，从这里开始可以去掉协方差矩阵 $\boldsymbol{\Sigma}_i$ ：将 $\boldsymbol{\Sigma}^{1/2}$ 定义为 $\boldsymbol{\Sigma}$ 的矩阵平方根，则平方马氏范数可以改写为：

其中

$$\|\mathbf{e}\|_{\boldsymbol{\Sigma}}^2 \triangleq \mathbf{e}^\top \boldsymbol{\Sigma}^{-1} \mathbf{e} = \left(\boldsymbol{\Sigma}^{-1/2} \mathbf{e} \right)^\top \left(\boldsymbol{\Sigma}^{-1/2} \mathbf{e} \right) = \left\| \boldsymbol{\Sigma}^{-1/2} \mathbf{e} \right\|_2^2. \quad (1.22)$$

因此，只需在式 (1.21b) 的每一项中，用 $\boldsymbol{\Sigma}_i^{-1/2}$ 左乘雅可比和预测误差，就可以消除协方差矩阵 $\boldsymbol{\Sigma}_i$ ：

$$\mathbf{A}_i = \boldsymbol{\Sigma}_i^{-1/2} \mathbf{H}_i \quad (1.23a)$$

$$\mathbf{b}_i = \boldsymbol{\Sigma}_i^{-1/2} (\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)). \quad (1.23b)$$

这个过程称为**白化**。例如，对于标量测量，它仅仅意味着将每一项除以测量标准差 σ_i 。注意，这一步消除了测量的单位（例如长度、角度），因此不同的行可以组合到同一个代价函数中。

1.3.2 将 SLAM 写成线性最小二乘

线性化后，最终得到如下标准最小二乘问题：

$$\delta^* = \arg \min_{\delta} \sum_i \|A_i \delta_i - b_i\|_2^2 \quad (1.24a)$$

$$= \arg \min_{\delta} \|A\delta - b\|_2^2. \quad (1.24b)$$

这里的 A 和 b 分别由收集所有白化雅可比矩阵 A_i 以及白化预测误差 b_i 得到，并将它们组合成一个大的矩阵 A 和右端项向量 b 。

雅可比矩阵 A 是一个规模很大但稀疏的矩阵，其分块结构与底层因子图的结构相对应。下面将详细考察这种稀疏结构。不过首先回顾经典线性代数方法，说明如何求解这个最小二乘问题。

1.3.3 用矩阵分解求解最小二乘

对于一个满列秩的 $m \times n$ 矩阵 A ($m \geq n$)，式 (1.24b) 的唯一最小二乘解可以通过求解正规方程获得：

$$(A^\top A) \delta^* = A^\top b. \quad (1.25)$$

通常的做法是分解信息矩阵（也称 Hessian 矩阵） Λ ：

$$\Lambda \triangleq A^\top A = R^\top R. \quad (1.26)$$

这里的 Cholesky 因子 R 是一个 $n \times n$ 的上三角矩阵⁴，通过 Cholesky 分解计算得到；Cholesky 分解是针对对称正定矩阵的 LU（下三角—上三角）分解变体。随后，先求解

$$R^\top y = A^\top b \quad (1.27)$$

得到 y ，再通过前代和回代求解

$$R\delta^* = y \quad (1.28)$$

得到 δ^* 。对于稠密矩阵，Cholesky 分解需要 $n^3/3$ 次浮点运算；整个算法（包括计算对称矩阵 $A^\top A$ 的一半）需要 $(m + n/3)n^2$ 次浮点运算。还可以使用 LDL^{top} 分解，这是 Cholesky 分解的一个变体，可以避免开平方根。

另一种比 Cholesky 分解更准确、数值稳定性更好的方法是 QR 分解。该方法不必显式计算信息矩阵 Λ ，而是直接对 A 及其右端项进行 QR 分解：

$$A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad \begin{bmatrix} d \\ e \end{bmatrix} = Q^\top b. \quad (1.29)$$

其中， Q 是 $m \times m$ 正交矩阵， $d \in \mathbb{R}^n$ ， $e \in \mathbb{R}^{m-n}$ ，而 R 是同一个上三角 Cholesky 三角因子。对于稠密矩阵 A ，通常按列从左到右计算 R 。对每一列 j ，通过用 Householder 反射矩阵 H_j 左乘 A ，将对角线下方的所有非零元素置零。经过 n 次迭代， A 完全分解为

$$\mathbf{H}_n \dots \mathbf{H}_2 \mathbf{H}_1 \mathbf{A} = \mathbf{Q}^\top \mathbf{A} = \begin{bmatrix} \mathbf{R} \\ \mathbf{0} \end{bmatrix}. \quad (1.30)$$

通常不会显式构造正交矩阵 $\mathbf{Q}$ ，而是将 $\mathbf{b}$ 作为额外一列附加到 $\mathbf{A}$ ，直接计算变换后的右端项 $\mathbf{Q}^\top \mathbf{b}$ 。由于 $\mathbf{Q}$ 是正交矩阵，有

$$\|\mathbf{A}\delta - \mathbf{b}\|_2^2 = \|\mathbf{Q}^\top \mathbf{A}\delta - \mathbf{Q}^\top \mathbf{b}\|_2^2 = \|\mathbf{R}\delta - \mathbf{d}\|_2^2 + \|\mathbf{e}\|_2^2. \quad (1.31)$$

其中使用了式 (1.29) 的等式。显然， $\|\mathbf{e}\|_2^2$ 就是最小二乘残差平方和，而最小二乘解 δ^* 可以通过回代求解三角系统

$$\mathbf{R}\delta^* = \mathbf{d} \quad (1.32)$$

得到。注意，QR 分解得到的上三角因子 $\mathbf{R}$ （对角线上的符号可能不同）与 Cholesky 分解得到的因子相同，因为

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} \mathbf{R} \\ \mathbf{0} \end{bmatrix}^\top \mathbf{Q}^\top \mathbf{Q} \begin{bmatrix} \mathbf{R} \\ \mathbf{0} \end{bmatrix} = \mathbf{R}^\top \mathbf{R}. \quad (1.33)$$

这里再次使用了 $\mathbf{Q}$ 为正交矩阵这一事实。QR 的计算成本主要来自 Householder 反射，为 $2(m - n/3)n^2$ 次浮点运算。与 Cholesky 相比，当 $m \gg n$ 时，两种算法都需要 $O(mn^2)$ 次运算，但 QR 分解大约慢两倍。

总之，与 SLAM 线性化优化问题相关的线性代数可以简洁地概括为：将信息矩阵 $\mathbf{A}$ 或测量雅可比矩阵 $\mathbf{A}$ 分解成平方根形式。由于它们都基于 SAM 问题中的矩阵平方根，我们将这类方法称为平方根 SAM，简称 SAM [249, 254]。

1.4 非线性优化

本节讨论求解式 (1.18) 所定义非线性最小二乘问题的一些经典优化方法。回顾一下，SLAM 中的非线性最小二乘目标函数为

$$J(\mathbf{x}) \triangleq \sum_i \|z_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2. \quad (1.34)$$

它对应于一个由测量以及部分或全部未知量的先验密度共同构成的非线性因子图。

一般来说，非线性最小二乘问题无法直接求解，而需要从合适的初始估计出发迭代求解。非线性优化方法通过依次求解式 (1.18) 的线性近似，逐渐逼近最小值^[264]。不同算法在局部近似代价函数的方式，以及根据这种局部近似寻找改进估计的方式上有所不同。关于非线性求解器的深入综述见^[814]；^[389] 则从线性代数角度进行讨论。

所有算法都具有如下基本结构：从初始估计 $\mathbf{x}^0$ 出发，在每次迭代中计算更新步 δ ，并将其应用到下一次估计 $\mathbf{x}^{t+1} = \mathbf{x}^t + \delta$ 。当达到某些收敛准则时停止，例如变化量 δ 的范数低于一个很小的阈值。

1.4.1 最速下降

最速下降 (steepest descent, SD) 使用当前估计处的最速下降方向来计算更新步：

$$\delta^{\text{sd}} = -\alpha \nabla J(\mathbf{x})|_{\mathbf{x}=\mathbf{x}^t}, \quad (1.35)$$

其中 $\mathbf{x}^t$ 是当前的 $\mathbf{x}$ 估计。这里使用负梯度确定最速下降方向。对于非线性最小二乘目标函数（式 1.34），在当前点附近将目标函数近似为二次函数 $J(\mathbf{x}) \approx \|\mathbf{A}(\mathbf{x} - \mathbf{x}^t) - \mathbf{b}\|_2^2$ ，在该线性化点可以得到精确梯度

$$\nabla J(\mathbf{x})|_{\mathbf{x}=\mathbf{x}^t} = -2\mathbf{A}^\top \mathbf{b}.$$

步长 α 必须谨慎选择，以平衡更新的安全性与合理的收敛速度。可以执行显式线搜索，在给定方向上寻找最小值。SD 算法简单，但在最小值附近收敛较慢。

1.4.2 高斯—牛顿

高斯—牛顿（Gauss—Newton, GN）通过使用二阶更新实现更快收敛。GN 利用非线性最小二乘问题的特殊结构，用 $\mathbf{A}^\top \mathbf{A}$ 近似 Hessian 矩阵。GN 更新步通过求解正规方程（式 1.25）得到：

$$\mathbf{A}^\top \mathbf{A} \delta^{\text{gn}} = \mathbf{A}^\top \mathbf{b}. \quad (1.36)$$

可以使用第 1.3.3 节中的任意方法求解。对于行为良好（即近似二次）的目标函数和较好的初始估计，高斯—牛顿表现出近似二次的收敛速度。如果二次近似很差，GN 步可能使新估计远离最小值，随后发生发散。

1.4.3 Levenberg—Marquardt

Levenberg—Marquardt (LM) 算法允许多次迭代直至收敛，同时控制我们愿意信任高斯—牛顿二次近似的区域。因此，这类方法通常称为**信赖域方法**。

为了结合 SD 和 GN 的优点，Levenberg^[646] 提议在正规方程（式 1.25）的对角线上加入非负常数 $\lambda \in \mathbb{R}^+ \cup \{0\}$ ：

$$(\mathbf{A}^\top \mathbf{A} + \lambda \mathbf{I}) \delta^{\text{lm}} = \mathbf{A}^\top \mathbf{b}. \quad (1.37)$$

当 $\lambda = 0$ 时得到 GN；当 λ 很大时，近似有 $\delta^* \approx \frac{1}{\lambda} \mathbf{A}^\top \mathbf{b}$ ，即沿代价函数 J （式 1.34）的负梯度方向更新。因此，LM 可以看作在 GN 和 SD 之间自然地进行折中。

随后 Marquardt^[731] 提议考虑对角元素的尺度，以获得更快的收敛：

$$(\mathbf{A}^\top \mathbf{A} + \lambda \text{diag}(\mathbf{A}^\top \mathbf{A})) \delta^{\text{lm}} = \mathbf{A}^\top \mathbf{b}. \quad (1.38)$$

这一修改会在梯度较小（目标函数近似平坦的方向）时沿最速下降方向采取更大的步长，因为此时对角元素的逆会较大；相反，在目标函数变化陡峭的方向上，算法会更加谨慎并采取更小的步长。对正规方程的这两种修改，都可以从贝叶斯角度解释为：给问题中的所有未知变量加入零均值先验。

GN 与 LM 的一个关键区别是，LM 会拒绝使残差平方和增大的更新。被拒绝意味着非线性函数在局部表现不佳，需要采取更小的步长。实现方式是启发式地增大 λ ，例如将当前值乘以 10，然后重新求解修改后的正规方程。另一方面，如果某一步使残差平方和降低，则接受该更新，并相应地更新状态估计；此时减小 λ （例如除以 10），然后以新的线性化点重复算法，直至收敛。

1.4.4 Dogleg 最小化

Powell 的 dogleg (PDL) 算法^[885] 可以作为 LM 的更高效替代方案^[697]。Levenberg—Marquardt 算法的一个主要缺点是：如果某一步被拒绝，就必须重新分解修改后的信息矩阵，而这正是算法中最昂贵的环节。因此，PDL 的核心思想是分别计算 GN 步和 SD 步，再将二者适当地组合起来。如果 LM 步被

拒绝，GN 步和 SD 步的方向仍然有效，可以用另一种方式组合，直到代价下降。因此，状态估计的每次更新只需进行一次矩阵分解，而不是多次。

图 1.6 展示了 GN 步和 SD 步的组合方式。组合步从 SD 更新开始，随后急转弯（因此称为 dogleg，即“狗腿”）朝向 GN 更新，但在信赖域边界处停止。与 LM 不同，PDL 显式维护一个信赖域，在该区域内我们相信线性假设。线性近似是否合适由增益比决定：

$$\rho = \frac{J(\mathbf{x}^t) - J(\mathbf{x}^t + \delta)}{L(\mathbf{0}) - L(\delta)}, \quad (1.39)$$

其中 $L(\delta) = \mathbf{A}^\top \mathbf{A} \delta - \mathbf{A}^\top \mathbf{b}$ 是在当前估计 $\mathbf{x}^t$ 处对式 (1.34) 非线性二次代价函数 J 的线性化。如果 ρ 很小（即 $\rho < 0.25$ ），说明代价下降幅度没有达到线性化预测，因而缩小信赖域。相反，如果实际下降与预测相符或更好（即 $\rho > 0.75$ ），则根据更新向量的大小扩大信赖域，并接受该步。

1.5 因子图与稀疏性

到目前为止介绍的求解器假设相关矩阵可以是稠密的；但在 SLAM 中，稠密方法无法扩展到实际问题规模。对于图 1.1 的玩具问题，稠密方法能够正常工作。图 1.5 所示的较大模拟问题更接近真实世界，不过对于 SLAM 而言它仍然相对较小，因为实际问题常常包含数千乃至数百万个未知量。我们之所以仍能处理这类问题，正是因为其稀疏性。

直接观察因子图即可认识到这种稀疏性。由图 1.5 可见，该图是稀疏的（远非完全连接图）。连接 100 个未知位姿的里程计链，是由 100 个二元因子构成的线性结构，而不是可能存在的 100^2 个（二元）因子。此外，对于 20 个路标，最多可以有 2000 个因子将每个路标与每个位姿相连，但实际数量更接近 400。最后，路标之间完全没有因子，这反映了我们没有获得它们相对位置的任何信息。这种结构是大多数 SLAM 问题的典型特征。

1.5.1 稀疏雅可比矩阵及其因子图

现代 SLAM 算法的关键在于利用稀疏性。SLAM 中因子图的一个重要性质是：它准确表示了最终稀疏雅可比矩阵 A 的稀疏分块结构。为此，重新考察非线性 SLAM 问题内循环中的关键计算——最小二乘问题：

$$\delta^* = \arg \min_{\delta} \sum_i \|\mathbf{A}_i \delta_i - \mathbf{b}_i\|_2^2. \quad (1.40)$$

上式每一项都来自原始非线性 SLAM 问题中的一个因子，并在当前线性化点附近进行线性化（式 1.21b）。矩阵 A_i 可以按变量拆分成若干分块，并收集成一个大的分块稀疏雅可比矩阵，其稀疏结构恰好由因子图给出。

尽管这些线性问题通常作为非线性优化的内部迭代出现，下面省略更新量的上标，因为相关结论对其来源无关，适用于一般线性问题。

考虑图 1.1 玩具示例的因子图。线性化后得到具有图 1.7 所示分块结构的稀疏系统 $[A \ b]$ 。将其与因子图进行比较可以发现：每个因子对应一个分块行，每个变量对应 A 的一个分块列。总共有 9 个分块行，分别对应因子分解 $\phi(p_1, p_2, p_3, \ell_1, \ell_2)$ 中的 9 个因子。

1.5.2 稀疏信息矩阵及其图

如第 1.3.3 节所述，使用 Cholesky 分解求解正规方程时，首先要形成 Hessian 矩阵或信息矩阵 $\mathbf{\Lambda} = A^\top A$ 。注意， $A^\top A$ 并不是真正的 Hessian，而是由残差的泰勒级数截断得到的“高斯—牛顿”近似。一般

$$\begin{bmatrix} \Lambda_{11} & & \Lambda_{13} & \Lambda_{14} & & \\ & \Lambda_{22} & & & \Lambda_{25} & \\ \Lambda_{31} & & \Lambda_{33} & \Lambda_{34} & & \\ \Lambda_{41} & & \Lambda_{43} & \Lambda_{44} & \Lambda_{45} & \\ & \Lambda_{52} & & \Lambda_{54} & \Lambda_{55} & \end{bmatrix}$$

图 1.6: 图 1.7 图 1.1 玩具 SLAM 示例的稀疏雅可比矩阵 A 的分块结构，其中空白项为零。

而言，由于雅可比 A 是分块稀疏的，Hessian Λ 也应当是稀疏的。按构造， Λ 是对称矩阵；当唯一 MAP 解存在时，它还是正定的。

信息矩阵 Λ 的稀疏模式自然定义了一个无向图 G ：如果两个变量曾经共同出现在同一个因子中，它们之间就存在一条边。在分块层面， $\Lambda = A^T A$ 的稀疏模式正好是该图的邻接矩阵。这一结论不限于二元因子：一个 n 元因子会在其全部变量之间诱导一个团，因此对应所有变量对之间的非零分块。

图 1.8 展示了 (a) 玩具示例信息矩阵 Λ 的分块结构，以及 (b) 对应的无向图 G 。下面经常使用与 Λ 关联的图 G 来讨论稀疏性和填充。

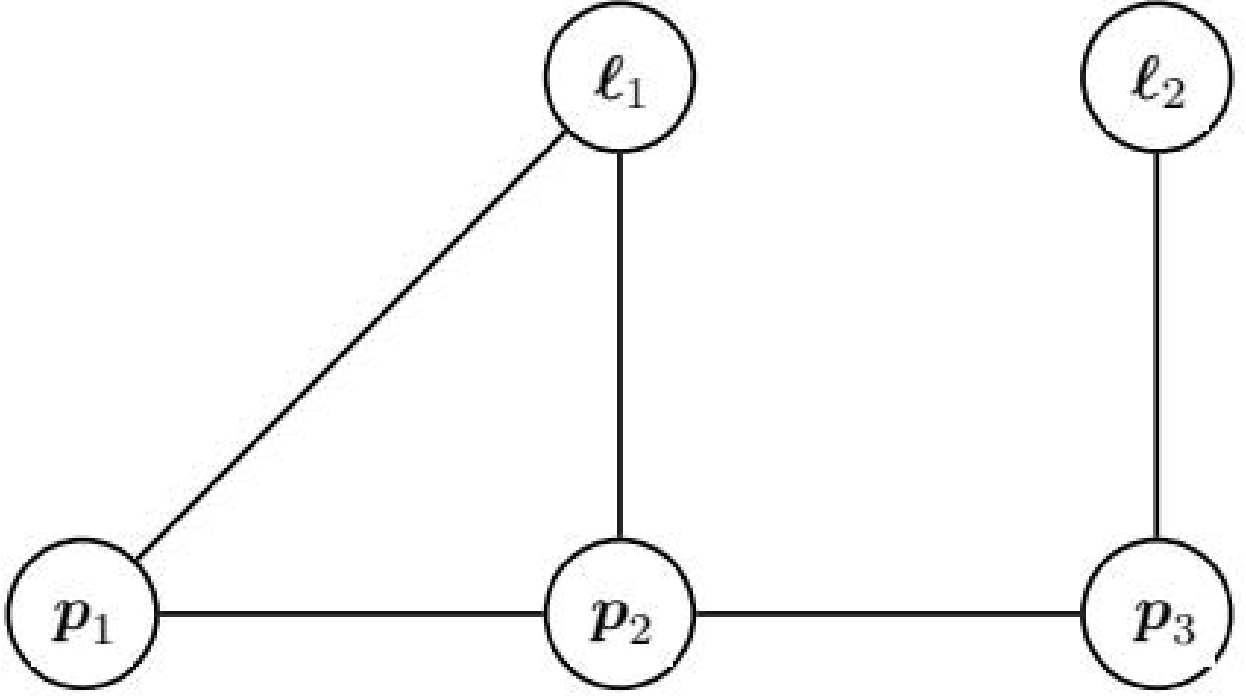
1.5.3 稀疏分解

我们已经看到，MAP 估计等价于求解第 1.3.3 节所述的线性方程组；对于非线性最小二乘问题，则在迭代过程中反复求解这类系统。前两节说明， A 和 $A^T A$ 都具有由因子图决定的稀疏性。无需展开细节，这种已知的稀疏模式可以显著加速 Cholesky 分解（处理 $A^T A$ 时）或 QR 分解（处理 A 时）。例如，CHOLMOD^[189] 和 SuiteSparseQR^[240] 都提供了高效实现，许多软件包在底层使用它们。实践中，在稀疏问题上，稀疏 Cholesky 或 LDU 分解通常也优于 QR 分解，而且优势并非只是一个常数因子。

稀疏分解所需的浮点运算量远低于稠密矩阵。关键在于，稀疏矩阵采用的列排序会显著影响总计算量。任何排序最终都会产生相同的 MAP 估计，但变量排序决定矩阵因子中的填充量（即超出待分解矩阵原有稀疏模式的额外非零项）。已知寻找使矩阵分解填充最小的变量排序是 NP-hard 问题^[1239]，因此必须采用良好的启发式方法，这将进一步影响分解算法的计算复杂度。

仍以图 1.5 所示的较大模拟示例说明这一点。图 1.9 展示了对应稀疏雅可比矩阵 A 的模式，并在右上角给出信息矩阵 $\Lambda = A^T A$ 的模式。图右侧还给出了两种不同排序下得到的上三角 Cholesky 因子 R 。两者都具有稀疏性，也都满足 $R^T R = A^T A$ （变量置换除外），但稀疏程度不同，而这正决定了分解 A 的代价。第一种排序最自然：先排列位姿，再排列路标，得到的稀疏 R 因子有 9399 个非零项。相比之下，右下角按列近似最小度（COLAMD）启发式方法^[29, 241] 重新排序后得到的 R 因子只有 4168 个非零项。然而，回代会为两种排序给出完全相同的解。

还可以使用预条件共轭梯度等工具迭代求解正规方程。在具有特殊稀疏模式的视觉 SLAM 中，幂迭代也已取得成功^[1171]。不过，对于大多数 SLAM 问题，稀疏分解仍然是首选方法，而且它还具有很好的图模型解释，下面将对此进行讨论。


 图 1.7: 图 1.8 玩具 SLAM 问题的信息矩阵 $\mathbf{A}$ 及其关联的无向图 G 。

1.6 消元

到目前为止，我们一直从线性代数角度解释 SLAM 推断。本节将视角扩展到更抽象的层次，直接用图模型思考推断问题。这最终会引出当前最先进的 SLAM 求解器：下一节中用于增量平滑与建图的贝叶斯树。

1.6.1 变量消元算法

存在一种通用算法，可以针对任意（最好是稀疏的）因子图，计算未知变量 $\mathbf{x}$ 上相应的后验密度 $p(\mathbf{x}|\mathbf{z})$ ，并将其表示为易于恢复 MAP 解的形式。如前所述，因子图以因子乘积的形式表示未归一化后验 $\phi(\mathbf{x}) \propto p(\mathbf{x}|\mathbf{z})$ ；在 SLAM 问题中，该图通常直接由测量生成。变量消元算法提供了一套规则，可将因子图转换为另一个称为**贝叶斯网络**的图模型，该模型只依赖未知变量 $\mathbf{x}$ ，从而可以方便地进行 MAP 推断（以及采样、边缘化等其他操作）。

具体来说，变量消元算法可以将如下形式的任意因子图

$$\phi(\mathbf{x}) = \phi(\mathbf{x}_1, \dots, \mathbf{x}_n) \quad (1.41)$$

分解为具有如下形式的贝叶斯网络概率密度：

$$p(\mathbf{x}) = p(\mathbf{x}_1 | \mathbf{s}_1) p(\mathbf{x}_2 | \mathbf{s}_2) \dots p(\mathbf{x}_n) = \prod_j p(\mathbf{x}_j | \mathbf{s}_j), \quad (1.42)$$

其中， $\mathbf{s}_j$ 表示在给定变量排序 $\mathbf{x}_1, \dots, \mathbf{x}_n$ 下，与变量 $\Delta \mathbf{x}_j$ 关联的分隔集 $\mathbf{s}(\mathbf{x}_j)$ 的一次赋值。分隔集定义为消元后 $\Delta \mathbf{x}_j$ 所依赖的变量集合。虽然这种分解类似链式法则，但对稀疏因子图进行消元通常会得到很小的分隔集。

消元算法每次消去一个变量 $\Delta \mathbf{x}_j$ ，从完整因子图 $\phi_{1:n}$ 开始。当消去每个变量时，生成一个条件密度 $p(\mathbf{x}_j | \mathbf{s}_j)$ ，以及只包含剩余变量的约简因子图 $\phi_{j+1:n}$ 。所有变量消元后，算法返回具有所需分解形式的贝

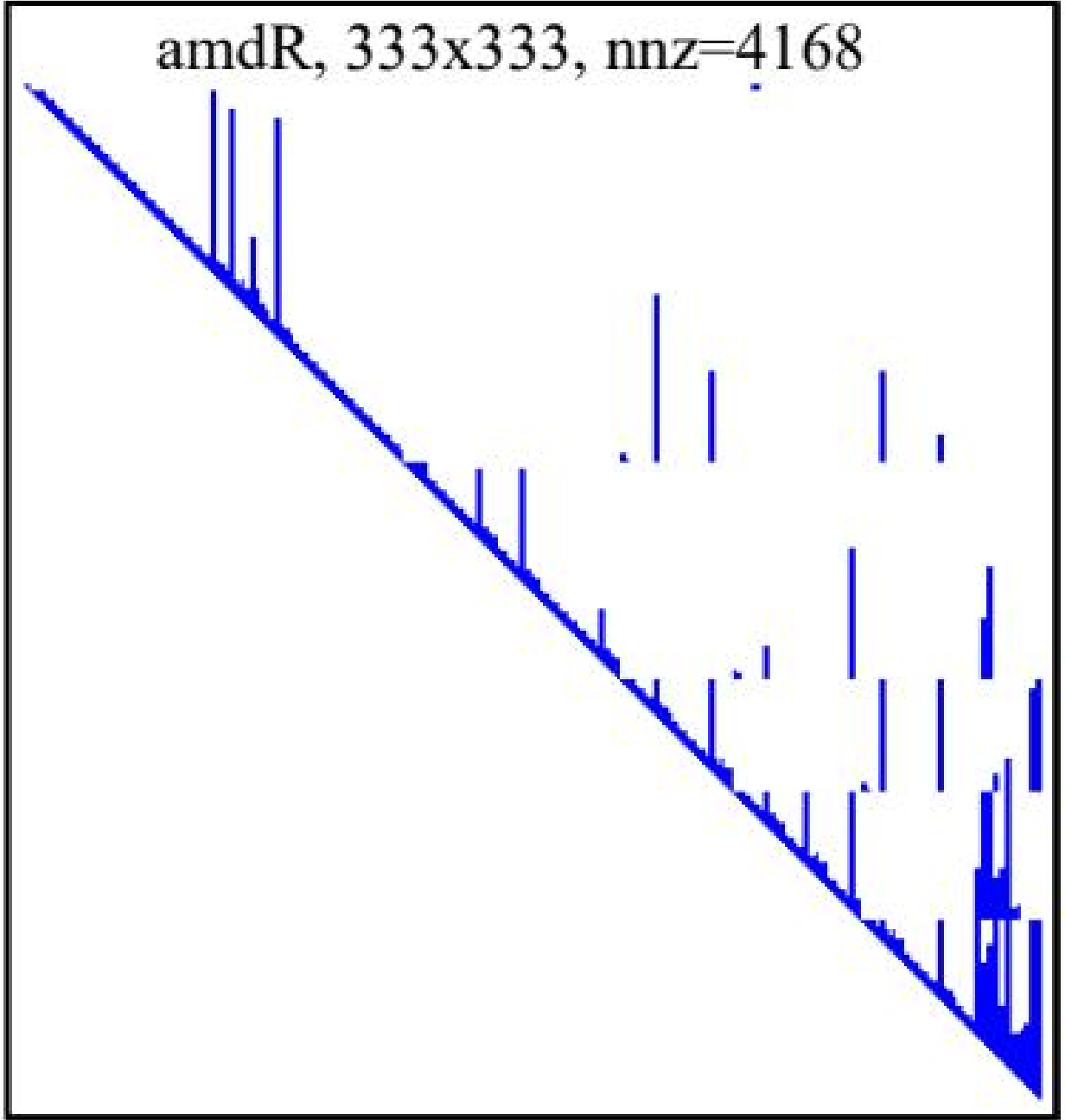


图 1.8: 图 1.9 左侧是图 1.5 问题对应的测量雅可比矩阵 A ；右侧从上到下依次为信息矩阵 $\Lambda = A^T A$ 、其上三角 Cholesky 因子 R ，以及采用更优变量排序（COLAMD）得到的替代因子 R 。

叶斯网络。

给定部分已消元的因子图 $\phi_{j:n}$ ，要消除单个变量 $\Delta \mathbf{x}_j$ ，首先移除所有与该变量相邻的因子 $\phi_i(\mathbf{x}_i)$ ，并将它们相乘得到乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 。随后把 ψ 分解为已消元变量上的条件分布 $p(\mathbf{x}_j | \mathbf{s}_j)$ 与分隔集上的新因子 $\tau(\mathbf{s}_j)$ ：

$$\psi(\mathbf{x}_j, \mathbf{s}_j) = p(\mathbf{x}_j | \mathbf{s}_j) \tau(\mathbf{s}_j). \quad (1.43)$$

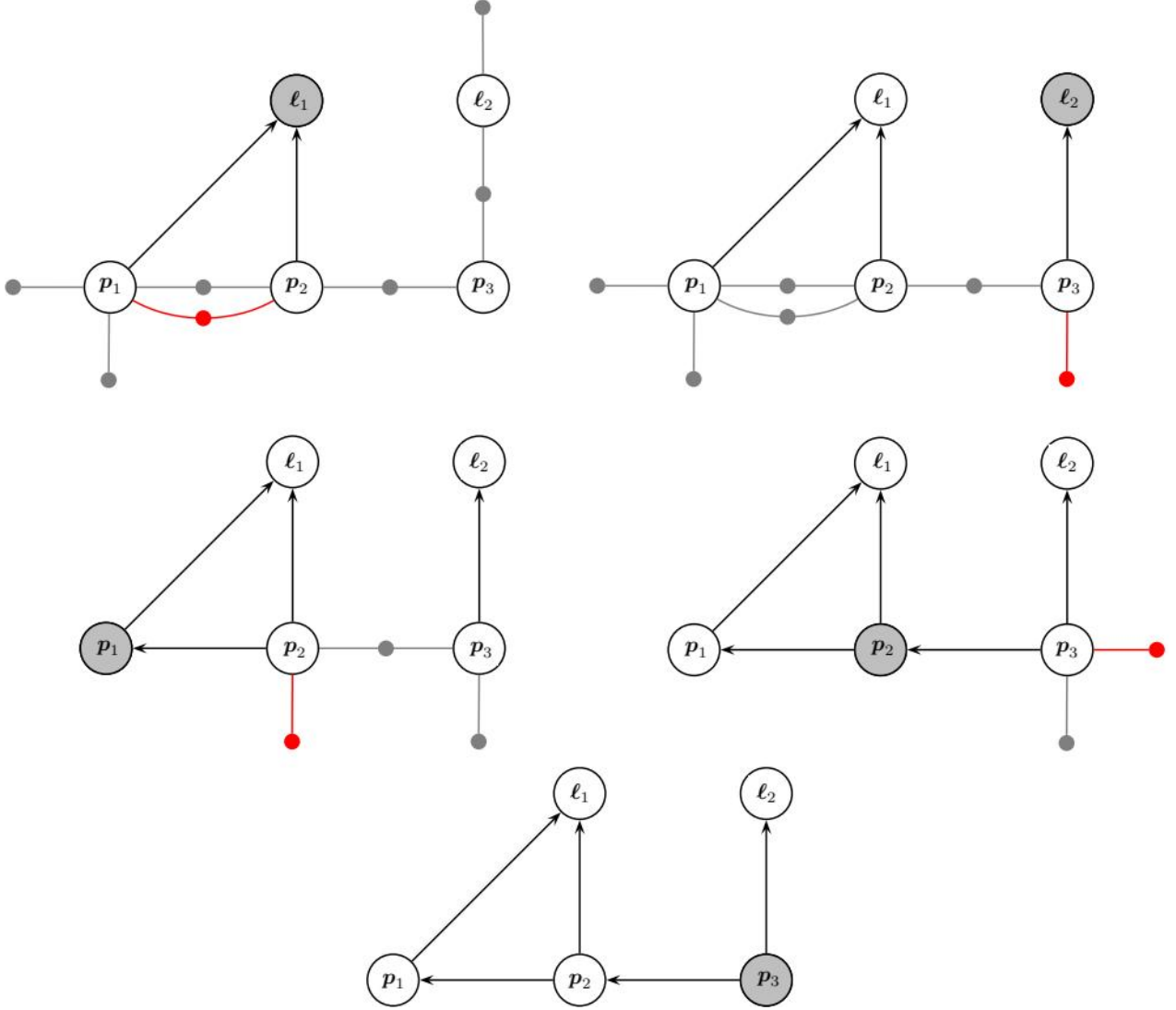


图 1.9: 图 1.10 玩具 SLAM 示例的变量消元：按 $\ell_1, \ell_2, p_1, p_2, p_3$ 的顺序，将图 1.3 的因子图转换为贝叶斯网络（右下）。

因此，从 $\phi(\mathbf{x})$ 到 $p(\mathbf{x})$ 的完整分解，可以看作连续执行 n 个局部分解步骤。当消除最后一个变量 $\mathbf{x}_n$ 时，分隔集 s_n 为空，因此得到的条件密度只是对 $\mathbf{x}_n$ 的先验 $p(\mathbf{x}_n)$ 。

对于玩具示例，一种可能的消元序列如图 1.10 所示，排序为 $\ell_1, \ell_2, p_1, p_2, p_3$ 。每一步中，被消元的变量以灰色阴影表示，分隔集 s_j 上新生成的因子 $\tau(s_j)$ 以红色表示。整体而言，变量消元算法将因子图 $\phi(\ell_1, \ell_2, p_1, p_2, p_3)$ 分解为图 1.10（右下）的贝叶斯网络，对应的分解为

$$p(\ell_1, \ell_2, p_1, p_2, p_3) = p(\ell_1 | p_1, p_2) p(\ell_2 | p_3) p(p_1 | p_2) p(p_2 | p_3) p(p_3). \quad (1.44)$$

1.6.2 线性高斯消元

在线性测量函数和加性正态分布噪声的情况下，变量消元算法等价于稀疏矩阵分解；稀疏 Cholesky 分解和 QR 分解都是该通用算法的特例。

如前所述，消元算法逐个处理变量。对于每个变量 $\Delta \mathbf{x}_j$ ，移除与之相邻的所有因子 $\phi_i(\mathbf{x}_i)$ ，并形成中间乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 。只需将所有矩阵 A_i 累积到一个更大的分块矩阵 $\bar{A}_j$ 中即可：

$$\psi(\mathbf{x}_j, \mathbf{s}_j) \leftarrow \prod_{i \in \mathcal{N}_j} \phi_i(\mathbf{x}_i) \quad (1.45a)$$

$$= \exp \left(-\frac{1}{2} \sum_i \|\mathbf{A}_i \mathbf{x}_i - \mathbf{b}_i\|_2^2 \right) \quad (1.45b)$$

$$= \exp \left(-\frac{1}{2} \|\bar{\mathbf{A}}_j [\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j\|_2^2 \right), \quad (1.45c)$$

其中，新的右端项向量 $\bar{\mathbf{b}}_j$ 堆叠了所有 $\mathbf{b}_i$ ；符号 $[\cdot; \cdot]$ 表示按列（垂直）堆叠。

考虑在玩具示例中消去变量 ℓ_1 。与它相邻的因子为 ϕ_4, ϕ_7, ϕ_8 ，从而诱导出分隔集 $\mathbf{s}_1 = [\mathbf{p}_1; \mathbf{p}_2]$ 。乘积因子等于

$$\psi(\ell_1, \mathbf{p}_1, \mathbf{p}_2) = \exp \left(-\frac{1}{2} \|\bar{\mathbf{A}}_1 [\ell_1; \mathbf{p}_1; \mathbf{p}_2] - \bar{\mathbf{b}}_1\|_2^2 \right), \quad (1.46)$$

其中

$$\bar{\mathbf{A}}_1 \triangleq \begin{bmatrix} \mathbf{A}_{41} & & \\ \mathbf{A}_{71} & \mathbf{A}_{73} & \\ \mathbf{A}_{81} & & \mathbf{A}_{84} \end{bmatrix}, \quad \bar{\mathbf{b}}_1 \triangleq \begin{bmatrix} \mathbf{b}_4 \\ \mathbf{b}_7 \\ \mathbf{b}_8 \end{bmatrix}. \quad (1.47)$$

观察图 1.7 的稀疏雅可比矩阵，这相当于取出第一列中存在非零分块的那些分块行，也就是与 ℓ_1 相邻的三个因子。

接下来可以用多种方式对乘积 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 进行分解。这里讨论 QR 变体，因为它与线性化因子的联系更加直接。具体而言，与乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 对应的增广矩阵 $[\bar{\mathbf{A}}_j | \bar{\mathbf{b}}_j]$ 可使用部分 QR 分解^[389] 重写为

$$[\bar{\mathbf{A}}_j | \bar{\mathbf{b}}_j] = \mathbf{Q} \begin{bmatrix} \mathbf{R}_j & \mathbf{T}_j & \mathbf{d}_j \\ & \tilde{\mathbf{A}}_\tau & \tilde{\mathbf{b}}_\tau \end{bmatrix}, \quad (1.48)$$

其中 $\mathbf{R}_j$ 是上三角矩阵。由此， $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 可以分解为

$$\begin{aligned} \psi(\mathbf{x}_j, \mathbf{s}_j) &= \exp \left\{ -\frac{1}{2} \|\bar{\mathbf{A}}_j [\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j\|_2^2 \right\} \\ &= \exp \left\{ -\frac{1}{2} \|\mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 \right\} \exp \left\{ -\frac{1}{2} \|\tilde{\mathbf{A}}_\tau \mathbf{s}_j - \tilde{\mathbf{b}}_\tau\|_2^2 \right\} \\ &= p(\mathbf{x}_j | \mathbf{s}_j) \tau(\mathbf{s}_j), \end{aligned} \quad (1.49a)$$

$$\|\bar{\mathbf{A}}_j [\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j\|_2^2 = \|\mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 + \|\tilde{\mathbf{A}}_\tau \mathbf{s}_j - \tilde{\mathbf{b}}_\tau\|_2^2. \quad (1.49b)$$

其中利用了旋转矩阵 $\mathbf{Q}$ 不会改变相关范数值这一事实。

在玩具示例中，图 1.11 展示了消去第一个变量 ℓ_1 （其分隔集为 $[p_1; p_2]$ ）的结果。图中同时展示了因子图上的操作，以及对图 1.7 稀疏雅可比矩阵的相应影响（省略右端项）。分界线以上的部分对应正在形成的稀疏上三角矩阵 $\mathbf{R}$ ；矩阵的新贡献用蓝色表示，新生成的因子用红色表示。为完整起见，图 1.12 展示了其余四个变量消元步骤，给出了 QR 分解如何在一个小型示例上端到端执行。最后一步展示了生成的贝叶斯网络与稀疏上三角因子 $\mathbf{R}$ 之间的等价性。

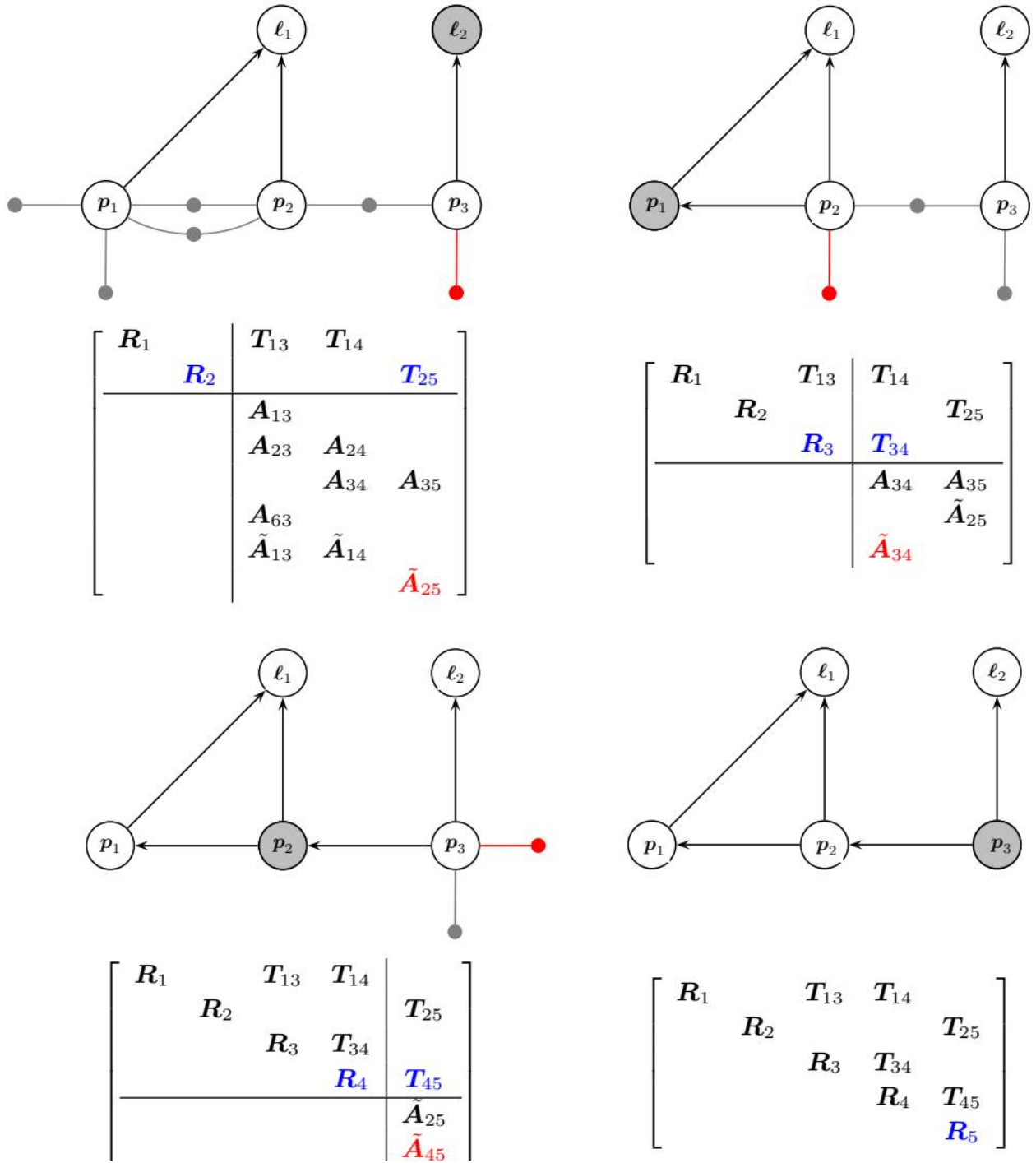


图 1.11: 图 1.12 玩具示例中剩余的消元步骤，完成一次完整 QR 分解。右下角的最后一步展示了最终贝叶斯网络与稀疏 Cholesky 因子 R 的等价性。

$$\|R_j \mathbf{x}_j + T_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 = (\mathbf{x}_j - \boldsymbol{\mu}_j)^\top R_j^\top R_j (\mathbf{x}_j - \boldsymbol{\mu}_j) \triangleq \|\mathbf{x}_j - \boldsymbol{\mu}_j\|_{\Sigma_j}^2, \quad (1.52)$$

其中，均值 $\boldsymbol{\mu}_j = R_j^{-1}(\mathbf{d}_j - T_j \mathbf{s}_j)$ 线性依赖于分隔集 s_j ，协方差矩阵为 $\Sigma_j = (R_j^\top R_j)^{-1}$ 。因此，归一化常数为 $k = |2\pi\Sigma_j|^{-1/2}$ 。

消元步骤完成后，使用回代获得每个变量的 MAP 估计。如图 1.12 所示，最后被消元的变量不依赖于任何其他变量，因此可以直接从贝叶斯网络中提取该变量的 MAP 估计。按照消元顺序的逆序继续进行，每个条件分布的所有分隔变量都将在前面的步骤中获得，从而可以计算当前前端变量的估计。

在每一步中，变量 $\Delta \mathbf{x}_j$ 的 MAP 估计就是条件均值：

$$\mathbf{x}_j^* = R_j^{-1}(\mathbf{d}_j - T_j \mathbf{s}_j^*), \quad (1.53)$$

因为到此时，分隔集 s_j 的 MAP 估计 s_j^* 已经完全确定。

1.7 增量式 SLAM

在增量式 SLAM 场景中，机器人穿行环境并不断接收新测量时，我们希望及时（或至少以固定间隔）计算最优轨迹和地图。一种方式是用新测量更新最近一次矩阵分解，复用已经纳入先前全部测量的计算结果。在线性情况下，可以通过增量分解方法实现这一点，其稠密版本在 Golub 和 Van Loan [389] 中有详细讨论。然而，矩阵分解作用于线性系统，而实际关心的大多数 SLAM 问题都是非线性的。采用增量矩阵分解时，如何在不重新分解完整矩阵的情况下执行增量式重新线性化，并不显然。为解决这个问题，我们再次借助图模型，引入一种新的图模型——贝叶斯树。随后将说明，如何在系统加入新测量和新状态时增量更新贝叶斯树，从而得到增量平滑与建图 (iSAM) 算法。

1.7.1 贝叶斯树

众所周知，在树结构图中的推断是高效的。相比之下，典型机器人问题对应的因子图包含许多环。不过，我们可以分两步构造树结构图模型：第一步，对因子图执行变量消元，得到具有特殊性质的贝叶斯网络；第二步，利用该特殊性质，找到该贝叶斯网络中各个团之间的树结构。

通过因子图运行消元算法得到的贝叶斯网络具有一个特殊性质：它是弦图 (chordal graph)，即任意长度大于 3 的无向环都包含一条弦，也就是连接环上两个非相邻顶点的边。在人工智能和机器学习领域，弦图更常称为三角化图。由于它仍然是贝叶斯网络，其联合密度 $p(\mathbf{x})$ 可以在各个变量 $\Delta \mathbf{x}_j$ 上分解：

$$p(\mathbf{x}) = \prod_j p(\mathbf{x}_j | \pi_j), \quad (1.54)$$

其中， π_j 是 $\Delta \mathbf{x}_j$ 的父节点。不过，尽管贝叶斯网络是弦图，在变量层面它仍是一个非平凡图：既不是链式结构，也不是树结构。图 1.10 最后一步给出了当前玩具 SLAM 示例的弦贝叶斯网络；其中存在无向环 $\mathbf{p}_1 - \mathbf{p}_2 - \ell_1$ ，说明它不具备树结构。

通过识别这个弦图中的团，可以将贝叶斯网络改写为贝叶斯树。我们引入这种新的树结构图模型，用来捕捉贝叶斯网络中的团结构。贝叶斯网络中的团之所以能构成一棵树，并不显然；这是弦性质的结果，这里不再证明。将所有这些团列在一棵无向树上，就得到团树，在人工智能和机器学习中也称为连接树。贝叶斯树只是团树的有向版本，同时保留了消元顺序的信息。

更形式化地说，贝叶斯树是一棵有向树，其节点表示底层弦贝叶斯网络中的团 c_k 。具体而言，每个节点定义一个条件密度 $p(f_k | s_k)$ ，其中分隔集 s_k 是团 c_k 与其父团 ϖ_k 的交集 $c_k \cap \varpi_k$ 。前端变量 f_k 是剩余变量，即 $f_k \triangleq c_k \setminus s_k$ 。记号上，将一个团写成 $c_k = f_k : s_k$ 。由贝叶斯树定义的变量联合密度为

$$p(\mathbf{x}) = \prod_k p(\mathbf{f}_k | \mathbf{s}_k). \quad (1.55)$$

对于根节点 f_r ，分隔集为空，即它只是根变量上的简单先验 $p(f_r)$ 。按照贝叶斯树的定义，团 c_k 的分隔集 $\mathbf{s}_k$ 始终是其父团 ϖ_k 的子集，因此图中的有向边与贝叶斯网络中的语义相同：表示条件依赖。

与当前玩具 SLAM 问题（图 1.3）关联的贝叶斯树如图 1.13 所示。根团 $\mathbf{c}_1 = \{\mathbf{p}_2, \mathbf{p}_3\}$ （蓝色）由 p_2 和 p_3 组成，并与另外两个团相交：绿色团 $\mathbf{c}_2 = \{\ell_1, \mathbf{p}_1\} : \mathbf{p}_2$ ，以及红色团 $\mathbf{c}_3 = \{\ell_2\} : \{p_3\}$ 。颜色还表示平方根信息矩阵 R 的行如何映射到不同团，以及贝叶斯树如何捕捉它们之间的独立关系。例如，绿色和红色行只在属于根团的变量上相交，正如预期。

1.7.2 更新贝叶斯树

增量推断可以看作对贝叶斯树进行简单编辑。这种视角能够更好地解释和理解原本抽象的增量矩阵分解过程，也允许我们以贝叶斯树的形式存储和计算平方根信息矩阵，从而得到具有明确语义的稀疏存储结构。

为了增量更新贝叶斯树，我们有选择地将贝叶斯树的一部分转换回因子图形式。加入新测量时，相当于加入一个新因子；例如，涉及两个变量的测量会产生新的二元因子 $\phi(\mathbf{x}_j, \mathbf{x}_{j'})$ 。此时，只有连接包含 $\mathbf{x}_j$ 和 $\mathbf{x}_{j'}$ 的团与根节点的路径会受到影响。这些团下方的子树不受影响，其他不包含这两个变量的子树也不受影响。因此，更新贝叶斯树时，只需将受影响部分转换回因子图，加入与新测量关联的新因子，然后使用任意方便的消元顺序重新消元这个临时因子图，构造新的贝叶斯树，最后重新接回未受影响的子树。

之所以只有树的上部受到影响，可以从贝叶斯树的两个重要性质理解。第一，贝叶斯树由弦贝叶斯网络按照消元顺序的逆序构成，因此每个团中的变量通过消去子团来收集来自子团的信息；任何团中的信息只会向上方根节点传播。第二，一个因子只有在与它相连的第一个变量被消去时才会进入消元过程。结合这两个性质可知，新因子不会影响那些不是其变量后继的变量。但是，如果一个因子涉及分别沿不同（即相互独立）路径连接到根的变量，那么为了表达它们之间新产生的依赖关系，就必须重新消去这些路径。

图 1.14 展示了如何在典型 SLAM 示例中应用这些增量分解/推断步骤。在该示例中，我们在 $\mathbf{p}_1$ 与 $\mathbf{p}_3$ 之间加入一个新因子，因此只影响树的左分支（左上图红色虚线标记部分）。随后构造右上图所示因子图：为每个团密度 $p(p_2, p_3)$ 和 $p(\ell_1, p_1 | p_2)$ 建立因子，并加入新因子 $f(p_1, p_3)$ 。右下图显示了按 ℓ_1, p_1, p_2, p_3 顺序消元后得到的图。最后，左下图给出重新组装的贝叶斯树，它由消元图得到的新树和原始贝叶斯树中未受影响的团（绿色）构成。

图 1.15 展示了一个小型 SLAM 序列的贝叶斯树示例，数据取自 Olson 等人^[824]著名的“曼哈顿世界”模拟序列第 400 步。机器人探索环境时，新测量通常只影响树的一小部分，因此只需重新计算这些部分。

1.7.3 增量平滑与建图

将上述内容结合起来，并考虑重新线性化等实际问题，就得到一种最先进的增量式非线性机器人 MAP 估计方法 iSAM。第一个版本 iSAM1^[531]使用了 Golub 和 Van Loan^[389]的增量矩阵分解方法，但它以次优方式处理线性化：定期在完整因子图上进行线性化，和/或在矩阵填充变得难以处理时进行线性化。第二个版本 iSAM2 使用贝叶斯树表示后验密度^[534]；随后如上所述，每当接收到新测量，就通过贝叶斯树增量更新来处理。

重新消除受影响团时，应该采用怎样的变量排序？只有贝叶斯树受影响部分中的变量需要更新。一种策略是在受影响变量上局部应用 COLAMD。但还可以做得更好：将最近访问的变量强制放到排序末端，即放入根团。对于这种增量变量排序策略，可以使用约束 COLAMD 算法^[241]。它既能将最近访问的

变量置于末端，又能保持总体上良好的排序。通常，后续更新只会影响树的一小部分，因此除大型回环外，大多数情况下都能高效完成。

更新树之后，还需要更新解。贝叶斯树中的回代从不依赖其他变量的根节点开始，逐步向叶节点进行。不过，通常不必重新计算所有变量的解：树的局部更新往往不会影响远处的变量。相反，在每个团处检查向下传播的变量估计变化，当该变化低于一个很小的阈值时即可停止。

引入贝叶斯树的动机，是增量求解非线性优化问题。为此，我们有选择地对包含某些变量的因子重新线性化，前提是这些变量偏离线性化点超过一个很小的阈值。与上面的树修改不同，此时必须重新处理包含受影响变量的所有团，不仅包括它们作为前端变量的情形，也包括作为分隔变量的情形。这样会影响树的更大部分，但在大多数情况下仍显著低于重建整棵树的代价。此外，我们还需要回到原始因子，而不能直接把团转换成因子图，这要求在消元时缓存某些量。完整的增量非线性算法 iSAM2 可参见^[534]。

iSAM1 和 iSAM2 已成功应用于许多不同的机器人估计问题，其中变量之间存在非平凡约束，变量数量可达数百万；后续章节还会进一步讨论。两种算法都在 GTSAM 库中实现，代码见 <https://github.com/borglab/gtsam>。

1.8 延伸阅读与近期趋势

希望深入了解因子图的读者，可以参阅 Dellaert 等人^[255] 的长篇文章。由于篇幅所限，本章相对简略，并弱化了一些高级概念，以尽可能降低入门门槛。不过，因子图和一般变量消元算法都非常强大，值得系统掌握。之后建议阅读 Kaess 等人^[534]，进一步理解贝叶斯树这一支撑 GTSAM 等现代求解器的核心概念。

如今，因子图范式可以处理流形上的状态变量，融合多种传感器，处理离群测量，甚至纳入深度学习方法的输出。本手册的其余部分将大量讨论这些趋势，读者可继续阅读以了解详情。

第二章 高级状态变量表示

作者： Timothy Barfoot、Frank Dellaert、Michael Kaess、Jose Luis Blanco-Claraco

上一章详细介绍了如何采用因子图范式建立并求解 SLAM 问题。为便于说明，我们有意避开了所估计状态变量的一些细节。本章重新审视状态变量的性质，并介绍现代 SLAM 形式化中普遍存在的两个重要主题。

首先，我们需要更好的工具来处理带有特定约束的状态变量；这些约束为变量定义了一个流形，因此在优化时必须特别处理。SLAM 中存在许多流形，最常见的是与机器人旋转相关的流形（尤其是三维旋转，但平面旋转也属于这一类）。

其次，状态变量的另一个方面来自时间本身的性质。上一章隐式地假设机器人以离散时间步在世界中运动。本章将引入轨迹的平滑连续时间表示，并讨论它们如何与因子图形式完全兼容。我们以 Barfoot^[47] 为主要参考，并采用 Sola 等人^[1022] 的简化记号。

2.0.1 流形上的优化

在某些机器人问题中，可以使用向量值未知量；但在大多数实际场景中，我们必须处理三维旋转以及其他非向量流形。粗略地说，流形是一个拓扑闭合的曲面（例如圆的周长或球面）；重要的是，在每个点附近，流形局部上都类似于欧氏空间。处理流形需要更精密的数学工具，以考虑其特殊结构。本节讨论如何在流形上执行优化，这将在上一章向量空间优化框架的基础上展开。作为示例，图 2.1 对球面流形 $\mathcal{M}$ 及其切空间 $T_x\mathcal{M}$ 进行了可视化；切空间可作为 $x \in \mathcal{M}$ 处用于优化的局部坐标系。

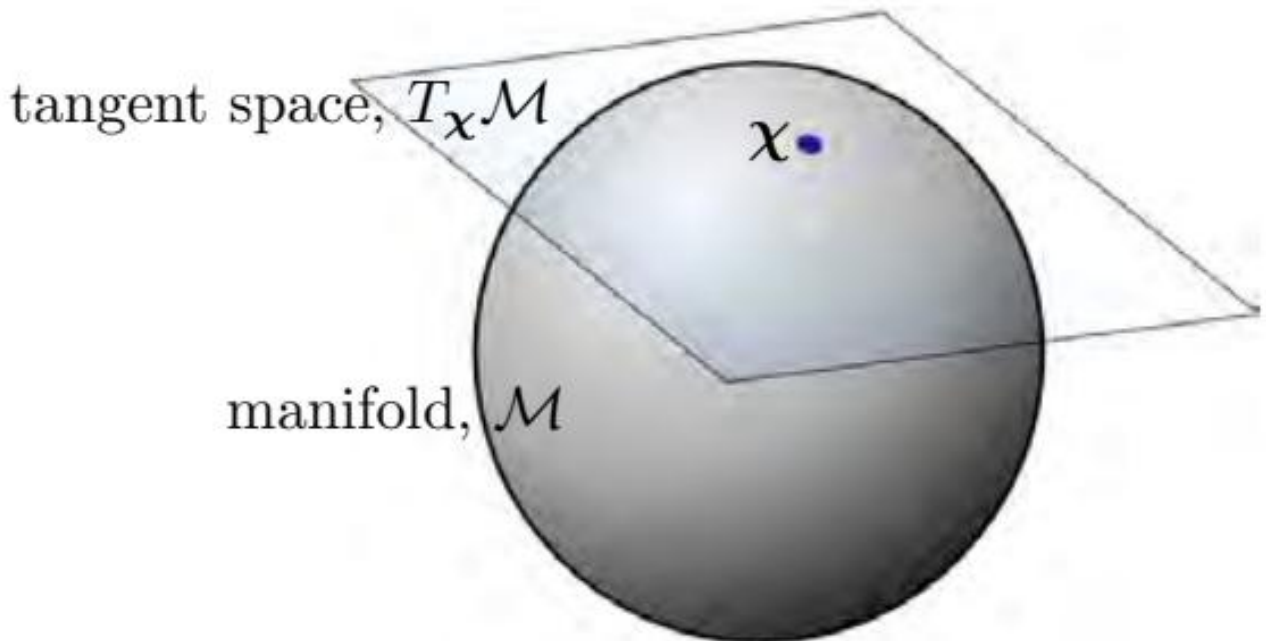


图 2.1: 图 2.1 对于球面流形 $\mathcal{M}$ ， $T_x\mathcal{M}$ 是局部切平面；配合局部基，可以定义局部坐标。

2.0.2 旋转与位姿

在 SLAM 语境中可以讨论许多流形，但最常见的两类用于表示旋转和位姿。旋转通常发生在二维（平面）或三维空间中，因此将旋转流形称为特殊正交群 $SO(d)$ ，其中 $d = 2$ 或 3 。平面旋转矩阵 $R_a^b \in SO(2)$ 的形式为

$$\mathbf{R}_a^b = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (2.1)$$

其中 $\theta \in \mathbb{R}$ 是旋转角，也是该情形下唯一的自由度。此外， R_a^b 可将参考坐标系 $\mathcal{F}^a$ 中表示的二维向量（例如路标）旋转到 $\mathcal{F}^b$ 中： $\ell^b = R_a^b \ell^a$ 。

三维旋转矩阵 $R_a^b \in SO(3)$ 同样用于在两个坐标系之间旋转向量，只是此时向量处于三维空间。三维旋转矩阵有 9 个元素，但只有 3 个自由度（例如滚转、俯仰和偏航）。二维和三维旋转矩阵都必须满足约束

$$\mathbf{R}_a^{b\top} \mathbf{R}_a^b = \mathbf{I}, \quad \det(\mathbf{R}_a^b) = 1,$$

以适当地限制自由度。

机器人的位姿同时包含旋转变量 $R_a^b \in SO(d)$ 和平移变量 $\mathbf{t}_a^b \in \mathbb{R}^d$ ，总共有 $3(d-1)$ 个自由度。有时我们分别跟踪这两类量，并使用 $\{R_a^b, \mathbf{t}_a^b\} \in SO(d) \times \mathbb{R}^d$ 表示位姿。另一种方式是将它们组合成一个 $(d+1) \times (d+1)$ 的变换矩阵：

$$\mathbf{T}_a^b = \begin{bmatrix} \mathbf{R}_a^b & \mathbf{t}_a^b \\ \mathbf{0} & 1 \end{bmatrix}. \quad (2.2)$$

所有这类变换矩阵组成的流形称为特殊欧氏群 $SE(d)$ ，其中 $d = 2$ 表示平面运动， $d = 3$ 表示三维运动。使用 $SE(d)$ 的好处是，可以通过一次矩阵乘法同时对路标进行旋转和平移：

$$\underbrace{\begin{bmatrix} \ell^b \\ 1 \end{bmatrix}}_{\tilde{\ell}^b} = \underbrace{\begin{bmatrix} \mathbf{R}_a^b & \mathbf{t}_a^b \\ \mathbf{0} & 1 \end{bmatrix}}_{\mathbf{T}_a^b} \underbrace{\begin{bmatrix} \ell^a \\ 1 \end{bmatrix}}_{\tilde{\ell}^a}, \quad (2.3)$$

其中 $\tilde{\ell}$ 是路标的齐次表示。

由于旋转矩阵和变换矩阵的形式受到约束，它们并不是向量。例如，不能简单地将两个旋转矩阵相加，并期望结果仍是有效的旋转矩阵。不过， $SO(d)$ 和 $SE(d)$ 都属于具有额外有用结构的流形，即**矩阵 Lie 群**。我们可以利用这种结构，在因子图 SLAM 中继续执行无约束 MAP 优化（参见 Dellaert 等人^[255]、Boumal^[106] 或 Barfoot^[47]）。关于机器人中的 Lie 群，可参考 Chirikjian 和 Kyatkin^[207] 以及 Chirikjian^[205, 206] 的经典工作。

2.0.3 矩阵 Lie 群

在 $SO(d)$ 和 $SE(d)$ 上执行优化的关键，是利用它们的群结构。例如，矩阵 Lie 群在矩阵乘法下封闭：若 $R_b^c, R_a^b \in SO(d)$ ，则乘积也属于该群，且 $R_a^c = R_b^c R_a^b \in SO(d)$ 。

矩阵 Lie 群还伴随一个非常有用的结构，称为 Lie 代数；它同时也是 Lie 群在单位元处的切空间。对我们而言，Lie 代数最重要的两个性质是：(i) 它是一个向量空间，其维度等于对应 Lie 群的自由度数量；(ii) 存在一个成熟的映射（矩阵指数）将 Lie 代数映射到 Lie 群。因此，可以从 Lie 代数元素相对容易地构造 Lie 群元素。例如，对于 $SO(2)$ ，暂时省略上下标，有

$$\mathbf{R} = \text{Exp}(\theta) = \exp(\theta^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} (\theta^\wedge)^n = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \in SO(2), \quad (2.4)$$

其中

$$\theta^\wedge = \begin{bmatrix} 0 & -\theta \\ \theta & 0 \end{bmatrix}, \quad \theta \in \mathbb{R}. \quad (2.5)$$

量 $\theta^\wedge$ 属于 Lie 代数 $\mathfrak{so}(2)$ ，通过矩阵指数 $\exp(\cdot)$ 映射为 Lie 群元素 R 。反向也可以使用矩阵对数和 vee 算子完成： $\theta = \text{Log}(R) = (\log R)^\vee$ 。vee 算子 $(\cdot)^\vee$ 是 wedge 算子 $(\cdot)^\wedge$ 的逆。

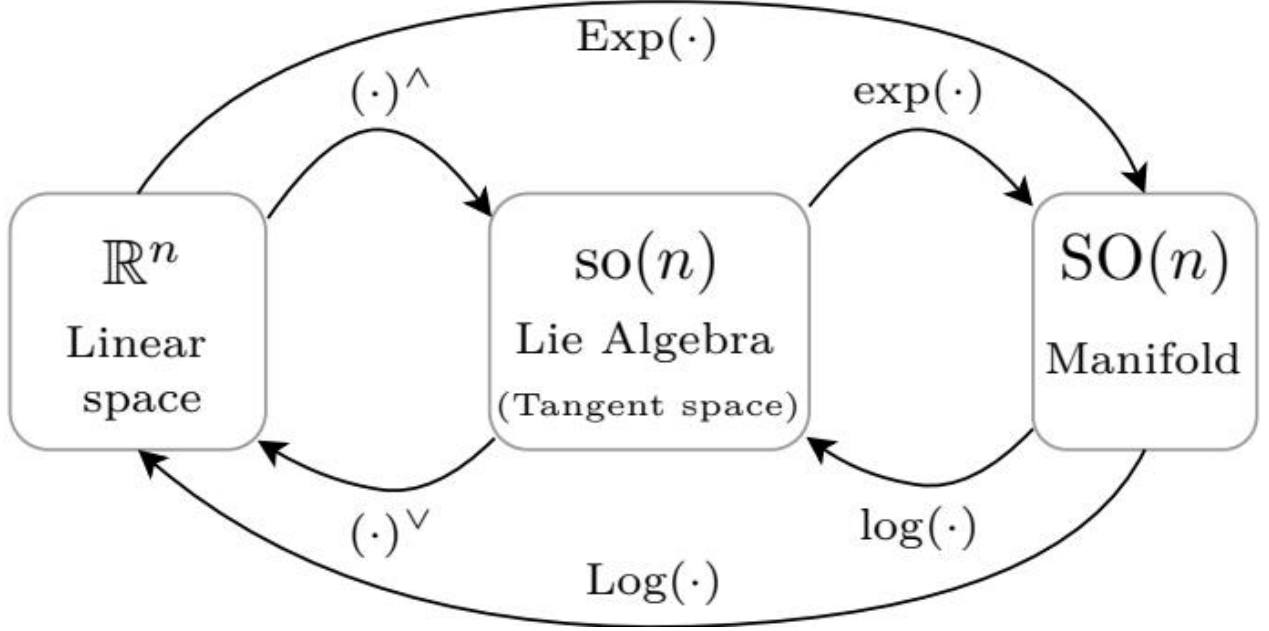


图 2.2: 图 2.2 本章记号概览：wedge 算子 $(\cdot)^\wedge$ 与 vee 算子 $(\cdot)^\vee$ ，指数映射 $\exp(\cdot)$ 与对数映射 $\log(\cdot)$ ，以及便捷记号 $\text{Exp}(\cdot) = \exp((\cdot)^\wedge)$ 和 $\text{Log}(\cdot) = (\log(\cdot))^\vee$ 。图中以 $\text{SO}(n)$ 为例，但这些算子对其他流形同样定义。

每个矩阵 Lie 群都有自己的线性 wedge 算子，用于从相应维度的标准向量空间构造 Lie 代数元素。对于 $\text{SO}(3)$ ，它是反对称算子：

$$\mathbf{R} = \text{Exp}(\boldsymbol{\theta}) = \exp(\boldsymbol{\theta}^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} \boldsymbol{\theta}^{\wedge n} \in \text{SO}(3), \quad (2.6a)$$

$$\boldsymbol{\theta}^\wedge = \begin{bmatrix} 0 & -\theta_3 & \theta_2 \\ \theta_3 & 0 & -\theta_1 \\ -\theta_2 & \theta_1 & 0 \end{bmatrix} \in \mathfrak{so}(3), \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} \in \mathbb{R}^3. \quad (2.6b)$$

对于 $\text{SE}(d)$ ，可写成

$$\mathbf{T} = \text{Exp}(\boldsymbol{\xi}) = \exp(\boldsymbol{\xi}^\wedge) \in \text{SE}(d), \quad (2.7a)$$

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\theta}^\wedge & \boldsymbol{\rho} \\ \mathbf{0} & 0 \end{bmatrix} \in \mathfrak{se}(d), \quad \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\theta} \end{bmatrix} \in \mathbb{R}^{3(d-1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{2d-3}, \quad \boldsymbol{\rho} \in \mathbb{R}^d. \quad (2.7b)$$

这里 $d = 2$ (平面) 或 3 (三维)。注意，wedge 算子的具体版本可以由输入向量的大小确定。对于这里讨论的每个矩阵 Lie 群，Lie 代数与 Lie 群之间的映射还存在众所周知的闭式表达式，可以替代矩阵指数的无穷级数形式^[47]。

2.0.4 Lie 群优化

建立矩阵 Lie 群之后，就可以利用它们将非线性最小二乘项“线性化”，从而执行 MAP 推断。回顾第 1.3.1 节，我们仍然要线性化测量函数 $\mathbf{h}_i(\cdot)$ ，只是其输入现在可能包含 Lie 群元素。

例如，假设 $\mathbf{h}_i(\cdot)$ 是一个相机模型，它接收在相机坐标系中表示的齐次路标 $\tilde{\ell}_i^c$ ，并返回图像中的像素坐标 $\mathbf{z}_i \in \mathbb{R}^2$ 。传感器生成模型可写为

$$\mathbf{z}_i = \mathbf{h}_i \left(\mathbf{T}_w^c \tilde{\ell}_i^w \right) + \boldsymbol{\eta}_i. \quad (2.8)$$

其中， $\mathbf{T}_w^c \in \text{SE}(3)$ 是相对于相机坐标系的世界坐标系位姿， $\tilde{\ell}_i^w$ 是在世界坐标系中表示的齐次路标， $\boldsymbol{\eta}_i$ 是通常的传感器噪声。我们希望求解

$$\mathbf{T}_w^{c*} = \arg \min_{\mathbf{T}_w^c} \sum_i \left\| \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}_w^c \tilde{\ell}_i^w \right) \right\|_{\Sigma_i}^2, \quad (2.9)$$

这就是透视 n 点 (perspective-n-point, PNP) 问题。本例假设世界坐标系中的路标位置已知；当然，在 SLAM 中也可能需要同时估计它们。

为了线性化传感器模型，利用 Lie 代数产生位姿的扰动：

$$\mathbf{T}_w^c = \mathbf{T}_w^{c^0} \text{Exp}(\boldsymbol{\xi}). \quad (2.10)$$

这里 $\boldsymbol{\xi} \in \mathbb{R}^6$ 产生一个“小的”位姿变化，用于扰动初始猜测 $\mathbf{T}_w^{c^0} \in \text{SE}(3)$ 。也常用简写记号

$$\mathbf{T}_w^c = \mathbf{T}_w^{c^0} \oplus \boldsymbol{\xi}, \quad (2.11)$$

它是式 (2.10) 的缩写。由于群的封闭性，这两个量的乘积必然属于 $\text{SE}(3)$ 。使用 Lie 代数定义位姿扰动，可以将其维度限制为三维位姿的实际自由度数量，从而在优化过程中无需额外引入约束。

还可以用矩阵指数的级数形式近似扰动位姿：

$$\mathbf{T}_w^c \approx \mathbf{T}_w^{c^0} (\mathbf{I} + \boldsymbol{\xi}^\wedge). \quad (2.12)$$

其中只保留了关于 $\boldsymbol{\xi}$ 的一次项。将式 (2.12) 代入测量函数 (式 2.8)，得到

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} (\mathbf{I} + \boldsymbol{\xi}^\wedge) \tilde{\ell}_i^w \right) + \boldsymbol{\eta}_i. \quad (2.13)$$

利用齐次点的线性算子 $\odot$ ，还可写成

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w + \mathbf{T}_w^{c^0} \tilde{\ell}_i^{w\odot} \boldsymbol{\xi} \right) + \boldsymbol{\eta}_i, \quad (2.14)$$

其中

$$\tilde{\ell}^\odot = \begin{bmatrix} \boldsymbol{\ell} \\ 1 \end{bmatrix}^\odot = \begin{bmatrix} \mathbf{I} & -\boldsymbol{\ell}^\wedge \\ \mathbf{0} & \mathbf{0} \end{bmatrix}. \quad (2.15)$$

现在还需对相机函数 $\mathbf{h}_i(\cdot)$ 进行线性化。利用一阶泰勒展开，有

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w \right) + \underbrace{\frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\ell}} \bigg|_{\mathbf{T}_w^{c^0} \tilde{\ell}_i^w}}_{\mathbf{H}_i \text{ (链式法则)}} \mathbf{T}_w^{c^0} \tilde{\ell}_i^{w\odot} \boldsymbol{\xi} + \boldsymbol{\eta}_i. \quad (2.16)$$

这里两部分相乘得到总体雅可比 $\mathbf{H}_i = \partial \mathbf{h}_i / \partial \boldsymbol{\xi}$ ，其链式法则结构十分清晰。也可以将传感器模型分解成若干基本操作，并使用中间向量状态逐步求导；对于本例，利用链式法则，关于相机位姿的雅可比可以拆成传感器模型、位姿作用、位姿复合和指数映射四项的乘积：

$$\mathbf{H}_i = \frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\xi}}, \quad (2.17)$$

各个单独雅可比都有已知表达式^[87]。注意，每个雅可比的线性化点都考虑了扰动发生在 $\boldsymbol{\xi} = \mathbf{0}$ 附近这一事实。

回到式 (1.21b)，该测量对应的线性化最小二乘项（即负对数因子）为

$$\left\| \left(\mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}_w^c \tilde{\ell}_i^w \right) \right) - \mathbf{H}_i \boldsymbol{\xi} \right\|_{\boldsymbol{\Sigma}_i}^2. \quad (2.18)$$

此时唯一未知量是位姿扰动 $\boldsymbol{\xi}$ 。将其与其他因子结合并求得状态变量的最优更新（包括 $\boldsymbol{\xi}^*$ ）后，需要按照式 (2.10) 的扰动方案更新初始猜测：

$$\mathbf{T}_w^c \leftarrow \mathbf{T}_w^c \oplus \boldsymbol{\xi}^*. \quad (2.19)$$

这样可以确保解始终位于 $\text{SE}(3)$ 中。优化照常迭代，直到所有状态更新（包括 $\boldsymbol{\xi}$ ）的变化足够小。式 (2.19) 适用于右侧位姿扰动；也可以选择左侧扰动，具体讨论见 Barfoot^[47]。

有时测量本身也位于流形上。例如，可能获得一个带噪位姿测量 $\mathbf{Z}_i$ ，其值属于 $\mathbf{T}_w^c \in \text{SE}(3)$ 。此时在 Lie 代数中构造误差：

$$\left\| \text{Log}(\mathbf{T}_w^c \mathbf{Z}_i^{-1}) \right\|_{\boldsymbol{\Sigma}_i}^2 \approx \left\| \text{Log}(\mathbf{T}_w^c \mathbf{Z}_i^{-1}) - \mathbf{H}_i \boldsymbol{\xi} \right\|_{\boldsymbol{\Sigma}_i}^2. \quad (2.20)$$

其中 $\mathbf{T}_w^c$ 按前述方式扰动， $\mathbf{H}_i$ 是该误差对应的雅可比^[47]。这样，任意类型的测量都可以在因子图范式中处理。

总之，对于属于 Lie 群的状态变量，可以通过上述方式执行无约束优化。关键是将测量函数相对于 Lie 代数中的扰动线性化，如式 (2.16) 所示。多数情况下可以解析地完成；也可以利用数值方法或自动微分计算所需雅可比。更一般地，这种针对 Lie 群元素函数的优化方法属于黎曼优化^[106]：Lie 代数也是流形的切空间，因而优化被限制在位姿（或旋转）流形的切方向上；按式 (2.19) 更新则相当于将更新量重新收回流形。黎曼优化同样适用于并非矩阵 Lie 群的流形，且除了矩阵指数外还可以使用其他回缩映射。

2.0.5 不确定性与 Lie 群

我们通常将估计中的不确定性表示为服从某种分布的随机状态变量。第 1.2.2 节已经讨论过，高斯分布最为常用。对于向量变量 $\mathbf{x}$ ，可写为

$$\mathbf{x} = \boldsymbol{\mu} + \boldsymbol{\delta}, \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad (2.21)$$

其中 $\boldsymbol{\mu}$ 是均值， $\boldsymbol{\Sigma}$ 是协方差矩阵，即将状态拆分为均值与零均值噪声之和。

对于 Lie 群，不能简单地将噪声加到旋转矩阵 $\mathbf{R} \in \text{SO}(d)$ 上，因为这样会破坏群的封闭性，结果不再是有效旋转矩阵。通常使用矩阵指数的满射映射，将高斯噪声 $\boldsymbol{\delta}$ 与“均值”旋转 $\bar{\mathbf{R}} \in \text{SO}(d)$ 组合：

$$\mathbf{R} = \bar{\mathbf{R}} \oplus \boldsymbol{\delta} = \bar{\mathbf{R}} \text{Exp}(\boldsymbol{\delta}), \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad (2.22)$$

这样可保证 $\mathbf{R} \in \text{SO}(d)$ 。对于 $\text{SE}(d)$ 同理：

$$\mathbf{T} = \bar{\mathbf{T}} \oplus \boldsymbol{\delta} = \bar{\mathbf{T}} \text{Exp}(\boldsymbol{\delta}), \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad (2.23)$$

此时可保证 $\mathbf{T} \in \text{SE}(d)$ ，并且 $\boldsymbol{\delta}$ 与 $\boldsymbol{\Sigma}$ 的维度必须相应匹配。这种做法本质上是在切空间中定义高斯分布，再通过指数映射将其诱导为旋转上的噪声分布。也可以直接在旋转上定义概率分布，某些情况下这样具有计算优势；第 6 章将再次讨论。

2.0.6 Lie 群的其他工具

关于 Lie 群还可以讨论很多内容^[1033, 106]。为保持简洁，这里只汇集本章及后续章节会用到的几个有用事实。关于 Lie 群元素函数求导的更多细节将在第 4.3 节给出。

2.0.6.1 $\oplus$ 与 $\ominus$ 算子

前面已经使用 $\oplus$ 算子将 Lie 代数向量与 Lie 群元素组合。对于 $\text{SE}(d)$ ，有

$$\mathbf{T} = \mathbf{T}^0 \oplus \boldsymbol{\xi} = \mathbf{T}^0 \text{Exp}(\boldsymbol{\xi}) = \mathbf{T}^0 \exp(\boldsymbol{\xi}^\wedge) \in \text{SE}(d). \quad (2.24)$$

还可以定义两个 Lie 群元素之间的“差”运算 $\ominus$ ：

$$\mathbf{T} = \mathbf{T}^0 \text{Exp}(\boldsymbol{\xi}) \implies \boldsymbol{\xi} = \mathbf{T} \ominus \mathbf{T}^0, \quad (2.25a)$$

$$\boldsymbol{\xi} = \mathbf{T} \ominus \mathbf{T}^0 = \text{Log}(\mathbf{T}^{0^{-1}} \mathbf{T}) = \log(\mathbf{T}^{0^{-1}} \mathbf{T})^\vee \in \text{se}(d). \quad (2.25b)$$

这些算子可以将相关运算的细节抽象出来。

2.0.6.2 逆

进行扰动时，有时需要扰动旋转或变换矩阵的逆。在 $\text{SE}(d)$ 中，直接有

$$(\mathbf{T}^0 \oplus \boldsymbol{\xi})^{-1} = \text{Exp}(\boldsymbol{\xi})^{-1} \mathbf{T}^{0^{-1}} = \text{Exp}(-\boldsymbol{\xi}) \mathbf{T}^{0^{-1}} = (-\boldsymbol{\xi}) \oplus \mathbf{T}^{0^{-1}}. \quad (2.26)$$

可以看到，扰动从右侧移到左侧，同时带有负号。

2.0.6.3 伴随

Lie 群的伴随表示，是将群元素视为其 Lie 代数线性变换的一种方式。对于 $\text{SO}(d)$ ，伴随表示与群本身相同，故略去细节。对于 $\text{SE}(d)$ ，伴随表示不同于群的基本表示，因此需要进一步说明。

给定位姿 $\mathbf{T}$ ， $\text{SE}(d)$ 的伴随映射将局部坐标系中的 Lie 代数元素 $\boldsymbol{\xi}^\wedge \in \text{se}(d)$ 变换为全局坐标系中的元素 $\boldsymbol{\epsilon}^\wedge \in \text{se}(d)$ ：

$$\boldsymbol{\epsilon}^\wedge \oplus \mathbf{T} = \mathbf{T} \oplus \boldsymbol{\xi}^\wedge, \quad (2.27)$$

其对应的内自同构（共轭）映射为

$$\boldsymbol{\epsilon}^\wedge = \text{Ad}_T \boldsymbol{\xi}^\wedge = \mathbf{T} \boldsymbol{\xi}^\wedge \mathbf{T}^{-1}. \quad (2.28)$$

等价地，也可以用伴随矩阵 $\text{Ad}(\mathbf{T})$ 将 $\boldsymbol{\xi} \in \mathbb{R}^6$ 线性变换到 $\mathbb{R}^6$ ：

$$\text{Ad}_T \boldsymbol{\xi}^\wedge = (\text{Ad}(\mathbf{T}) \boldsymbol{\xi})^\wedge. \quad (2.29)$$

称 $\text{Ad}(\mathbf{T})$ 为 $\mathbf{T}$ 处 $\text{SE}(d)$ 的伴随表示。若

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix},$$

则

$$\text{Ad}(\mathbf{T}) = \begin{bmatrix} \mathbf{R} & \mathbf{t}^\wedge \mathbf{R} \\ \mathbf{0} & \mathbf{R} \end{bmatrix}. \quad (2.30)$$

伴随矩阵的一个用途，是把已知变换两侧的扰动互换：

$$\mathbf{T} \text{Exp}(\boldsymbol{\xi}) = \text{Exp}(\text{Ad}(\mathbf{T})\boldsymbol{\xi}) \mathbf{T}. \quad (2.31)$$

这一等式不需要任何近似。

2.0.6.4 雅可比

每个 Lie 群都有相应的雅可比，用于联系群元素的变化与其 Lie 代数元素。以 $\text{SO}(d)$ 为例，旋转矩阵 $\mathbf{R}_a^b$ 与角速度 $\boldsymbol{\omega}^b \in \mathbb{R}^{2d-3}$ 之间的运动学方程（Poisson 方程）为

$$\dot{\mathbf{R}}_a^b = \boldsymbol{\omega}^{b^\wedge} \mathbf{R}_a^b. \quad (2.32)$$

这里 $\boldsymbol{\omega}^b$ 表示坐标系 $\mathcal{F}^a$ 相对于 $\mathcal{F}^b$ 的角速度，并在 $\mathcal{F}^b$ 中表达。若将 $\mathbf{R}_a^b$ 参数化为 $\exp(\boldsymbol{\theta})$ ，则等价地有

$$\dot{\boldsymbol{\theta}} = \mathbf{J}^{-1}(\boldsymbol{\theta})\boldsymbol{\omega}^b, \quad (2.33)$$

其中 $\mathbf{J}(\boldsymbol{\theta})$ 是 $\text{SO}(d)$ 的（左）雅可比。它在组合矩阵指数乘积时非常有用。例如，当 $\boldsymbol{\theta}_1$ 足够小时，

$$\text{Exp}(\boldsymbol{\theta}_1)\text{Exp}(\boldsymbol{\theta}_2) \approx \text{Exp}(\boldsymbol{\theta}_2 + \mathbf{J}(\boldsymbol{\theta}_2)^{-1}\boldsymbol{\theta}_1). \quad (2.34)$$

其级数形式为

$$\mathbf{J}(\boldsymbol{\theta}) = \sum_{n=0}^{\infty} \frac{1}{(n+1)!} (\boldsymbol{\theta}^\wedge)^n. \quad (2.35)$$

闭式表达式可参阅 Barfoot^[47]。下文也用 $\mathbf{J}(\boldsymbol{\xi})$ 表示 $\text{SE}(d)$ 的左雅可比，具体含义由上下文决定。Lie 群雅可比也是利用可微神经网络估计位姿的核心工具（见第 4.2 节）。

2.1 连续时间轨迹

连续时间轨迹可以表示平滑的机器人运动。到目前为止，我们假设需要估计轨迹上的一系列离散位姿；但机器人通常在世界中相当平滑地运动，因此有必要采用更平滑的轨迹表示。连续时间轨迹主要有两类：参数化方法将已知的时间基函数组合成平滑轨迹，通常选择具有局部支撑的基函数（例如分段多项式/样条），从而保持因子图稀疏；非参数化方法利用核函数获得更强的表示能力，具体可以使用以时间为自变量的一维高斯过程（GP）表示轨迹。选择合适且具有物理动机的核函数后，GP 关联的因子图同样可以保持稀疏。

连续时间轨迹不仅能保证平滑性，在处理高频传感器（如 IMU，见第 11.2 节）和/或异步传感器时尤其有用。若每收到一次测量就向因子图加入一个机器人位姿，那么高频传感器或不同传感器的异步采样会迅速产生难以处理的因子图。连续时间方法可以用远少于测量数量的变量表示轨迹，从而保持问题可处理。这对于旋转式激光雷达、雷达以及卷帘快门相机等存在运动畸变的传感器特别有价值，因为可以使用每个点或像素的精确时间戳，将其与该时刻的轨迹关联起来。

完成 MAP 推断后，连续时间轨迹还允许在任意感兴趣时刻查询轨迹，而不仅限于测量时刻；可以进行插值，也可以谨慎地外推。这对 SLAM 输出的下游使用者很有帮助。将测量时刻、估计变量时刻与查询时刻分离，是参数化和非参数化连续时间方法的主要优势。

2.1.1 样条

参数化连续时间轨迹方法的思想，是使用 K 个已知时间基函数 $\Psi_k(t)$ 的加权和表示位姿：

$$\mathbf{p}(t) = \sum_{k=1}^K \Psi_k(t) \mathbf{c}_k. \quad (2.36)$$

其中 $\mathbf{c}_k$ 是未知系数。暂时仍在向量空间中讨论，后面再介绍 Lie 群上的实现。基函数通常选用样条，即分段多项式（例如 B 样条和三次 Hermite 多项式）。样条具有局部支撑，在其局部影响区域之外取零，因此非常有利。图 2.3 展示了这一设置：在任意时刻最多只有 4 个基函数非零，由此得到稀疏因子图。

与前面的离散时间方法相比，主要区别是使用系数变量而不是位姿变量；但这与一般因子图方法完全兼容。当在时刻 t_i 观测到路标 ℓ 时，传感器模型为

$$\mathbf{z}_i = \mathbf{h}_i(\mathbf{p}(t_i), \ell) + \boldsymbol{\eta}_i. \quad (2.37)$$

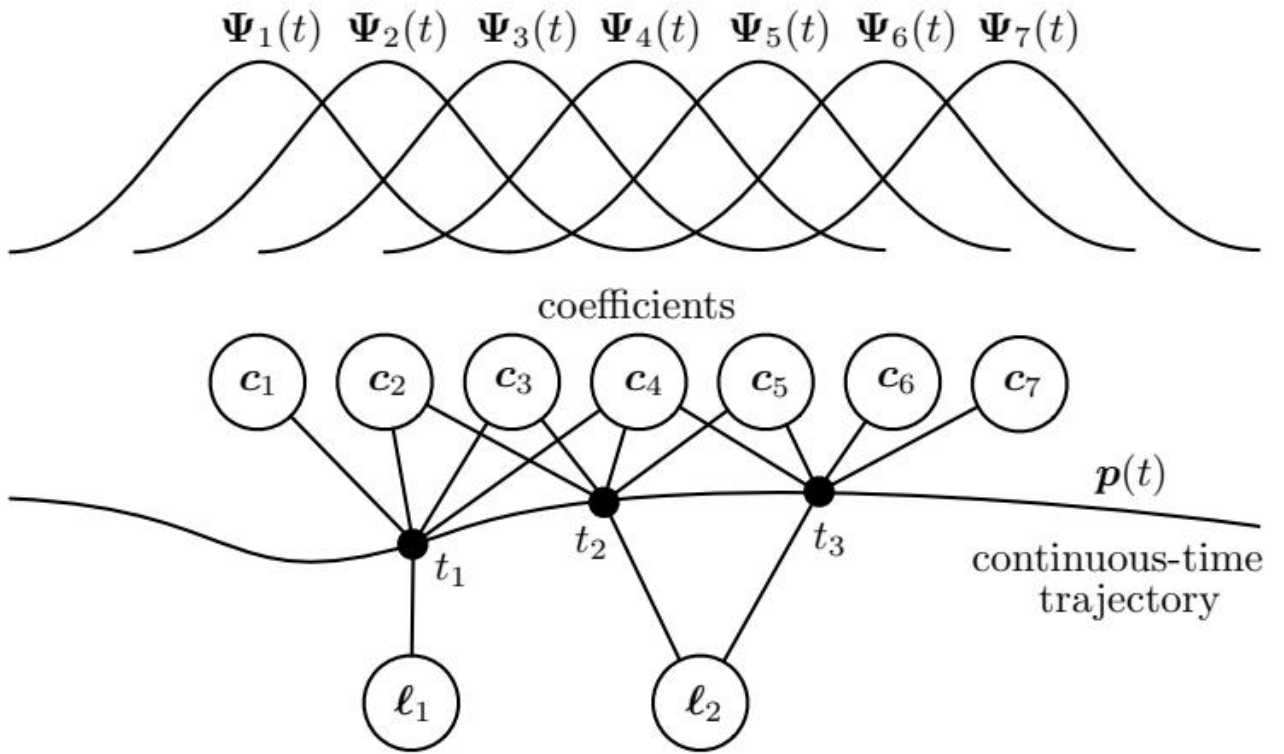


图 2.3: 图 2.3 用参数化样条表示连续时间轨迹。任意时刻的位姿由具有局部支撑的已知基函数加权求和得到；每个路标测量对应一个连接 4 个系数变量和 1 个路标变量的五元因子，整体因子图仍然非常稀疏。

代入式 (2.36)，有

$$\mathbf{z}_i = \mathbf{h}_i \left(\sum_{k=1}^K \Psi_k(t_i) \mathbf{c}_k, \ell \right) + \boldsymbol{\eta}_i. \quad (2.38)$$

如果基函数具有局部支撑，则在 t_i 处只有一小部分系数处于激活状态。令 $\mathbf{x}_i = [\mathbf{c}_i^\top \ell^\top]^\top$ 表示 t_i 处激活的系数变量以及路标变量，就重新得到

$$\mathbf{z}_i = \mathbf{h}_i(\mathbf{x}_i) + \boldsymbol{\eta}_i. \quad (2.39)$$

于是可以使用一般方法构造非线性最小二乘问题并进行优化。

如果基函数足够光滑，还可以方便地求取轨迹导数：

$$\dot{\mathbf{p}}(t) = \sum_{k=1}^K \dot{\Psi}_k(t) \mathbf{c}_k. \quad (2.40)$$

这样就能处理依赖速度甚至更高阶导数的传感器输出，而优化变量仍是同一组系数；只需计算基函数的相应导数即可。

通过 MAP 推断求得最优系数后，可以利用式 (2.36) 或式 (2.40) 在任意时刻查询轨迹及其导数。如果在推断过程中计算了系数估计的协方差（例如通过求信息矩阵的逆），还可以轻松将其映射为查询位姿或导数的协方差，因为式 (2.36) 和式 (2.40) 是线性关系；而基函数的局部支撑意味着只需使用相应的系数边缘协方差。

2.1.2 从参数化到非参数化

基本参数化连续时间方法的主要挑战，是必须决定基函数的类型和数量。基函数太多，容易对测量数据过拟合；太少，又可能没有足够能力表示真实轨迹形状，从而得到过度平滑的解。转向非参数化方法可以部分解决这一问题。

为简化说明，本节暂时假设没有路标变量，只关注位姿变量。采用上一节的参数化方法，线性化最小二乘项（负对数因子）为

$$\|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \Psi_i \delta_{c,i}\|_{\Sigma_i}^2. \quad (2.41)$$

其中， $\mathbf{x}_i^0$ 是当前解（激活的系数）， $\delta_{c,i}$ 是激活系数的更新量， Ψ_i 是在 t_i 处求值并堆叠后的激活基函数，雅可比为

$$\mathbf{H}_i = \left. \frac{\partial \mathbf{h}_i}{\partial \mathbf{x}} \right|_{\mathbf{x}_i^0}. \quad (2.42)$$

像以前一样收集各项，可以写成

$$\delta_c^* = \arg \min_{\delta_c} (\|\mathbf{b} - \mathbf{A} \Psi \delta_c\|^2 + \|\delta_c\|^2). \quad (2.43)$$

这里加入了正则项 $\|\delta_c\|^2$ ，倾向于使解的描述长度保持合理，即偏好接近零的样条系数，从而帮助避免过拟合。最优解满足

$$(\Psi^\top \mathbf{A}^\top \mathbf{A} \Psi + \mathbf{I}) \delta_c^* = \Psi^\top \mathbf{A}^\top \mathbf{b}. \quad (2.44)$$

这可以求得系数的最优更新 δ_c^* 。但我们真正关心的是位姿估计，而不是作为中间量的样条系数。测量时刻位姿变量的最优更新为 $\delta^* = \Psi \delta_c^*$ 。经过简单代数运算，可得

$$(\mathbf{A}^\top \mathbf{A} + \mathbf{K}^{-1}) \delta^* = \mathbf{A}^\top \mathbf{b}, \quad (2.45)$$

这是对式 (1.25) 正规方程的修改版本。核矩阵 $\mathbf{K} = \Psi \Psi^\top$ 起到正则化或平滑函数的作用。式 (2.45) 看起来比式 (2.44) 包含更大的线性系统，因为位姿数量多于样条系数。不过，非参数方法可以利用内置的插值能力，最终减少实际要求解的系统规模。

为了摆脱显式基函数，可以使用所谓的核技巧，用所选核函数 $\mathcal{K}(t, t')$ （例如平方指数核）的计算结果替代基函数的显式内积。构造核矩阵只需要计算基函数内积，因此令 $\mathbf{K} = [\mathcal{K}(t_i, t_j)]_{ij}$ 即可。此时不再估计样条的系数（参数），因而称为非参数方法；但仍需调节所选核函数的超参数（例如平方指数核的长度尺度），以实现期望的轨迹平滑程度。由于式 (2.45) 需要核矩阵的逆，直接这样做似乎代价很高；下一节将说明如何选择核函数，使逆核矩阵非常稀疏。

2.1.3 高斯过程

下面构造一族经过专门设计的核函数，使逆核矩阵及其对应因子图保持稀疏。上一节说明，可以用核函数替换基函数，从而得到非参数连续时间方法；但若直接进行替换，逆核矩阵可能变得稠密。这里从另一条思路出发：图 2.4 给出了一个由本节思想得到的因子图示例，它既保持稀疏，又产生平滑轨迹。

首先选择一个由白噪声驱动的线性时不变随机微分方程 (SDE)：²

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{L}\mathbf{w}(t), \quad (2.46)$$

其中 $\mathbf{w}(t) = \mathcal{GP}(\mathbf{0}, \mathbf{Q}\delta(t-t'))$ 是零均值白噪声高斯过程， $\mathbf{Q}$ 是功率谱密度矩阵， $\delta(\cdot)$ 是 Dirac delta 函数。该 SDE 将充当运动先验。它可以写出闭式积分解：

$$\mathbf{x}(t) = \Phi(t, t_1)\mathbf{x}(t_1) + \int_{t_1}^t \Phi(t, s)\mathbf{L}\mathbf{w}(s)ds, \quad (2.47)$$

其中 $\Phi(t, s) = \exp(\mathbf{A}(t-s))$ 称为转移函数， t_1 是第一次测量的时间戳。若初始状态均值为零，则后续状态均值也保持为零；状态的协方差函数（即核函数）为

$$\mathcal{K}(t, t') = \Phi(t, t_1)\mathcal{K}(t_1, t_1)\Phi(t', t_1)^\top + \int_{t_1}^{\min(t, t')} \Phi(t, s)\mathbf{L}\mathbf{Q}\mathbf{L}^\top\Phi(t', s)^\top ds. \quad (2.48)$$

在全部测量时刻计算核函数并构造核矩阵时，可以利用简洁关系

$$\mathbf{K} = \Phi\mathbf{Q}\Phi^\top, \quad (2.49)$$

其中 $\mathbf{Q} = \text{diag}(\mathcal{K}(t_1, t_1), Q_1, \dots, Q_M)$ ，且

$$\Phi = \begin{bmatrix} \mathbf{I} & & & & \\ \Phi(t_2, t_1) & \mathbf{I} & & & \\ \Phi(t_3, t_1) & \Phi(t_3, t_2) & \mathbf{I} & & \\ \vdots & \vdots & \vdots & \ddots & \\ \Phi(t_M, t_1) & \Phi(t_M, t_2) & \cdots & \Phi(t_M, t_{M-1}) & \mathbf{I} \end{bmatrix}. \quad (2.50)$$

其中

$$Q_i = \int_{t_{i-1}}^{t_i} \Phi(t_i, s)\mathbf{L}\mathbf{Q}\mathbf{L}^\top\Phi(t_i, s)^\top ds.$$

矩阵 Φ 是由各时间点之间的转移函数构成的下三角块矩阵，其对角块为单位阵。由于我们需要的是逆核矩阵， $\mathbf{K}^{-1} = \Phi^{-\top}\mathbf{Q}^{-1}\Phi^{-1}$ ；而 $\mathbf{Q}$ 是块对角矩阵，逆可以逐个对角块计算。更重要的是， Φ^{-1} 只有主块对角线和紧邻其下的一个块对角线非零：

$$\Phi^{-1} = \begin{bmatrix} \mathbf{I} & & & & \\ -\Phi(t_2, t_1) & \mathbf{I} & & & \\ & -\Phi(t_3, t_2) & \mathbf{I} & & \\ & & -\Phi(t_4, t_3) & \ddots & \\ & & & \ddots & \mathbf{I} \\ & & & & -\Phi(t_M, t_{M-1}) & \mathbf{I} \end{bmatrix}. \quad (2.51)$$

因此，任意长度轨迹的逆核矩阵 $\mathbf{K}^{-1}$ 都是块三对角的。根据前面对因子图的讨论，式 (2.45) 左端的稀疏性与因子图结构密切相关；这里 $\mathbf{K}^{-1}$ 作为整条轨迹的运动先验，可以用非常稀疏的因子图表示。

K^{-1} 之所以具有这种稀疏因子图，是因为出发点 SDE 的状态具有马尔可夫性。实际含义是：根据式 (2.46) 想表达的运动先验，状态可能需要包含更高阶量，而不只是位姿，还包括其导数。例如，若希望采用“恒速”先验，可以选择

$$\underbrace{\begin{bmatrix} \dot{\mathbf{p}}(t) \\ \dot{\mathbf{v}}(t) \end{bmatrix}}_{\dot{\mathbf{x}}(t)} = \underbrace{\begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \mathbf{p}(t) \\ \mathbf{v}(t) \end{bmatrix}}_{\mathbf{x}(t)} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{I} \end{bmatrix}}_{\mathbf{L}} \mathbf{w}(t). \quad (2.52)$$

这里的状态包含位姿及其导数。这种使用增广状态的形式有时称为同时轨迹估计与建图 (STEAM)，是 SLAM 的一种变体。

上述连续时间轨迹估计实际上是高斯过程回归^[909] 的一个例子。建立这一联系后，在测量时刻求解状态，就可以利用 GP 插值在其他时刻查询轨迹的均值和协方差。采用稀疏核方法时，每次查询的代价相对于测量数量 M 为常数，因为只涉及夹住查询时刻的两个估计状态。

还可以利用 GP 插值减少所需控制点的数量，不必在每个测量时刻都设置控制点，这类似于 GP 的诱导点思想。例如，每次激光扫描设置一个控制点，但仍使用扫描过程中每个点的独立时间戳。这样，测量时刻、估计时刻和查询时刻可以彼此不同。GP 的核还提供了适当的正则化，因此无需担心设置过多估计时刻导致过拟合，但仍需有足够的估计时刻来捕捉轨迹细节。

2.1.4 Lie 群上的样条与高斯过程

当状态位于流形上时，同样可以使用样条和 GP 连续时间方法。若流形是 Lie 群，两种方法都利用 Lie 代数，但方式不同。下面先介绍样条，再介绍高斯过程。

2.1.4.1 Lie 群上的样条

使样条适用于 Lie 群的关键，是采用累积形式。对于向量空间，假设所有自由度使用相同基函数，式 (2.36) 可简化为

$$\mathbf{p}(t) = \sum_{k=1}^K \psi_k(t) \mathbf{p}_k, \quad (2.53)$$

其中 $\mathbf{p}_k$ 是样条控制点， $\psi_k(t)$ 是标量基函数。将其改写为累积形式：

$$\mathbf{p}(t) = \psi_1^c(t) \mathbf{p}_1 + \sum_{k=2}^K \psi_k^c(t) (\mathbf{p}_k - \mathbf{p}_{k-1}), \quad (2.54)$$

其中累积基函数为

$$\psi_k^c(t) = \sum_{\ell=k}^K \psi_\ell(t). \quad (2.55)$$

例如，对时间间隔均匀为 T 的线性插值，定义 $\alpha_k(t) = (t - (k-1)T)/T$ ，普通基函数为

$$\psi_1(t) = \begin{cases} 1 - \alpha_1(t), & 0 \leq t < T \\ 0, & \text{其他情况,} \end{cases} \quad \psi_K(t) = \begin{cases} \alpha_K(t), & (K-1)T \leq t < KT \\ 0, & \text{其他情况,} \end{cases} \quad (2.56)$$

而对 $k = 2, \dots, K-1$,

$$\psi_k(t) = \begin{cases} \alpha_{k-1}(t), & (k-2)T \leq t < (k-1)T \\ 1 - \alpha_k(t), & (k-1)T \leq t < kT \\ 0, & \text{其他情况.} \end{cases} \quad (2.57)$$

对应的累积基函数为

$$\psi_1^c(t) = 1, \quad \psi_K^c(t) = \begin{cases} 0, & t < (K-1)T \\ \alpha_K(t), & (K-1)T \leq t, \end{cases} \quad (2.58)$$

以及对 $k = 2, \dots, K-1$,

$$\psi_k^c(t) = \begin{cases} 0, & t < (k-1)T \\ \alpha_k(t), & (k-1)T \leq t < kT \\ 1, & kT \leq t. \end{cases} \quad (2.59)$$

图 2.5 展示了这些基函数的形状。

累积基函数的主要优点是：在给定时间戳处，大多数基函数都不激活。对于线性样条，当 $(k-1)T \leq t < kT$ 时，

$$\mathbf{p}(t) = \mathbf{p}_{k-1} + \psi_k^c(t)(\mathbf{p}_k - \mathbf{p}_{k-1}). \quad (2.60)$$

此时只需计算一个基函数；更高阶样条在任意时刻也只会少量激活基函数。

在 Lie 群上使用样条时，用 Lie 群运算（矩阵乘法）替代累积表达式中的加法。以线性样条为例，在 $(k-1)T \leq t < kT$ 时，SE(d) 中的元素可以写成

$$\mathbf{T}(t) = \text{Exp}(\psi_k^c(t) \text{Log}(\mathbf{T}_k \mathbf{T}_{k-1}^{-1})) \mathbf{T}_{k-1}. \quad (2.61)$$

这里选择了左侧扰动，当然也可以采用右侧扰动。将 $\mathbf{T}(t_i)$ 代入任意测量表达式，在控制点 $\mathbf{T}_k$ 的 Lie 代数扰动上进行线性化，即可用于 MAP 框架。由于基函数具有紧支撑，在一个测量时刻只有少量控制点激活。

对于线性样条，式 (2.61) 可写为

$$\mathbf{T}(t) = (\mathbf{T}_k \mathbf{T}_{k-1}^{-1})^{\alpha_k(t)} \mathbf{T}_{k-1}. \quad (2.62)$$

对 $\mathbf{T}(t)$ 中的表达式进行线性化时，分别扰动两个控制位姿，使

$$\text{Exp}(\boldsymbol{\xi}(t)) \mathbf{T}^0(t) = \left(\text{Exp}(\boldsymbol{\xi}_k) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \text{Exp}(-\boldsymbol{\xi}_{k-1}) \right)^{\alpha_k(t)} \text{Exp}(\boldsymbol{\xi}_{k-1}) \mathbf{T}_{k-1}^0. \quad (2.63)$$

插值位姿扰动 $\boldsymbol{\xi}(t)$ 与两个控制点扰动 $\boldsymbol{\xi}_k$ 、 $\boldsymbol{\xi}_{k-1}$ 的关系可近似为

$$\boldsymbol{\xi}(t) \approx (\mathbf{I} - \mathbf{A}(\alpha_k(t))) \boldsymbol{\xi}_{k-1} + \mathbf{A}(\alpha_k(t)) \boldsymbol{\xi}_k, \quad (2.64)$$

其中

$$\mathbf{A}(\alpha_k(t)) = \alpha_k(t) \mathbf{J} \left(\alpha_k(t) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \right) \mathbf{J} \left(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \right)^{-1}. \quad (2.65)$$

这里 $\mathbf{J}(\cdot)$ 是 SE(d) 的左雅可比。利用式 (2.64)，即可将测量时刻位姿的变化与两侧控制点位姿关联起来，形成用于 MAP 估计的线性化误差。例如，式 (2.16) 的线性化测量模型可写成

$$\mathbf{e}_i(t) \approx \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}^0(t) \tilde{\ell}_i \right) - \mathbf{H}_i \boldsymbol{\xi}(t). \quad (2.66)$$

将式 (2.64) 代入即可得到关于两侧控制点的线性化误差：

$$\mathbf{e}_i(t) \approx \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}^0(t) \tilde{\ell}_i \right) - \mathbf{H}_i (\mathbf{I} - \mathbf{A}(\alpha_k(t))) \boldsymbol{\xi}_{k-1} - \mathbf{H}_i \mathbf{A}(\alpha_k(t)) \boldsymbol{\xi}_k. \quad (2.67)$$

其中还需将 t 时刻的名义位姿 $\mathbf{T}^0(t) = (\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}})^{\alpha_k(t)} \mathbf{T}_{k-1}^0$ 代入 h_i 和 H_i 。这相当于沿线性样条链式传播导数；高阶样条也可以采用同样过程。

2.1.4.2 Lie 群上的高斯过程

在 Lie 群上使用高斯过程时，同样利用 Lie 代数。图 2.6 展示了相应的 GP 运动先验因子。根据所选运动先验，控制点状态也可以包含额外的轨迹导数。

我们在一组控制点状态之间使用局部 GP，这与样条类似。对 $\text{SE}(d)$ ，定义控制点之间的局部变量 $\xi_k(t)$ 。例如，对 $\text{SE}(d)$ 的“随机游走”先验，可以选择

$$\dot{\xi}_k(t) = \mathbf{w}(t), \quad \mathbf{w}(t) = \mathcal{GP}(\mathbf{0}, \mathbf{Q}\delta(t - t')), \quad (2.68)$$

这里的变量定义在控制点 $\mathbf{T}_k$ 和 $\mathbf{T}_{k+1}$ 之间。该 SDE 的转移函数为单位阵，随机积分得到

$$\xi_k(t) = \underbrace{\xi_k(t_k)}_{\mathbf{0}} + \int_{t_k}^t \mathbf{w}(s) ds, \quad (2.69)$$

于是运动先验为

$$\xi_k(t) \sim \mathcal{GP}(\mathbf{0}, \min(t, t') \mathbf{Q}). \quad (2.70)$$

若控制点每隔 T 秒均匀分布，则逆核矩阵为 $\kappa^{-1} = \Phi^{-\top} \mathbf{Q}^{-1} \Phi^{-1}$ ，其中

$$\Phi^{-1} = \begin{bmatrix} \mathbf{I} & & & \\ -\mathbf{I} & \mathbf{I} & & \\ & \ddots & \ddots & \\ & & -\mathbf{I} & \mathbf{I} \end{bmatrix}, \quad \mathbf{Q} = \text{diag}(\mathcal{K}(t_1, t_1), T\mathbf{Q}, \dots, T\mathbf{Q}). \quad (2.71)$$

局部变量下的误差为

$$\mathbf{e}_k = \begin{cases} \text{Log}(\mathbf{T}_1 \check{\mathbf{T}}_1^{-1}) & k = 1 \\ \xi_{k-1}(t_k) - \xi_{k-1}(t_{k-1}) & k > 1 \end{cases}, \quad (2.72)$$

其中 $\check{\mathbf{T}}_1$ 是初始位姿先验。用全局变量表示时，同一误差为

$$\mathbf{e}_k = \begin{cases} \text{Log}(\mathbf{T}_1 \check{\mathbf{T}}_1^{-1}) & k = 1 \\ \text{Log}(\mathbf{T}_k \mathbf{T}_{k-1}^{-1}) & k > 1 \end{cases}. \quad (2.73)$$

图 2.6 给出了这种随机游走 GP 运动先验的因子图。与线性样条类似，若要在其他时刻查询轨迹，可以使用 GP 插值；对于随机游走先验，结果再次是线性插值：

$$\mathbf{T}(t) = (\mathbf{T}_k \mathbf{T}_{k-1}^{-1})^{\alpha_k(t)} \mathbf{T}_{k-1}, \quad (2.74)$$

其中 $\alpha_k(t) = (t - (k-1)T)/T$ ，且 $(k-1)T \leq t < kT$ 。与样条方法不同，这里的线性插值是 SDE 选择的间接结果，而不是显式指定的形式；如果一开始选择高阶 SDE，就会得到高阶插值样条。

最后，需要将误差项线性化以用于 MAP 估计。再次利用 Lie 群扰动方法，对于式 (2.73) 的第二种情形，有

$$\begin{aligned} \mathbf{e}_k &= \text{Log}(\text{Exp}(\xi_k) \mathbf{T}_k \mathbf{T}_{k-1}^{0^{-1}} \text{Exp}(-\xi_{k-1})) \\ &\approx \text{Log}(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}}) + \xi_k - \text{Ad}(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}}) \xi_{k-1}, \end{aligned} \quad (2.75)$$

其中 $\mathbf{T}_k^0$ 和 $\mathbf{T}_{k-1}^0$ 是当前猜测, $\boldsymbol{\xi}_k$ 和 $\boldsymbol{\xi}_{k-1}$ 是待求扰动, $\text{Ad}(\cdot)$ 是 $\text{SE}(d)$ 的伴随表示。式 (2.75) 的线性化形式可以在每次迭代中直接纳入标准 MAP 估计框架。

此外, 若想利用式 (2.74) 减少随机游走示例中的控制点数量, 可以采用式 (2.67) 中针对线性样条的同一方法, 因为两者最终都归结为 Lie 群控制点之间的线性插值。样条与 GP 的根本区别在于: GP 方法使用运动先验项 (见图 2.6) 对问题进行正则化, 而样条方法不使用这种先验。⁴

2.2 延伸阅读与近期趋势

关于流形, 尤其是 Lie 群上的估计, 已有大量研究。机器人领域的奠基性参考是 Chirikjian^[205] 及 Chirikjian 和 Kyatkin^[207]; 这些著作理论基础扎实, 并给出了处理流形不确定性的通用工具, 包括全局不确定性。关于流形优化, Boumal^[106] 是面向机器人应用的优秀参考。Barfoot^[47] 更详细地讨论了不确定性仍较为集中的 Lie 群估计, 这与本章内容相近。连续时间估计方面, Talbot 等人^[1067] 提供了参数化和非参数化方法的综合综述; Barfoot^[47] 也对非参数方法作了较为详细的说明。

与上一章类似, 离群点处理、使算法可微, 以及利用本章工具支持不同类型的传感器等近期趋势, 将在后续章节中进一步讨论。

后续章节

本项目的目录和构建脚本支持继续添加后续章节的中文翻译。